

Smart Teacher — File-by-File Deep Dive (Part 2)

Suite de [SMART_TEACHER_FILE_BY_FILE.md](#) — couvre les fichiers non encore détaillés : agentic graphs complets, handlers WebSocket, audio (STT/TTS), services, observability, database, RAG ingestion, et knowledge graph.

Sommaire

Agentic — Q&A graph (détaillé) 1. [agentic/state.py](#) 2. [agentic/qa/graph.py](#) 3. [agentic/qa/intent.py](#) 4. [agentic/qa/rewriter.py](#) 5. [agentic/qa/retriever.py](#) 6. [agentic/qa/responder.py](#) 7. [agentic/qa/reviewer.py](#)

Agentic — Teaching graph 8. [agentic/teaching/graph.py](#) 9. [agentic/teaching/planner.py](#) 10. [agentic/teaching/context.py](#) 11. [agentic/teaching/adaptation.py](#) 12. [agentic/teaching/narrator.py](#) 13. [agentic/teaching/reviewer.py](#) 14. [agentic/teaching/fallback.py](#)

Agentic — Resilience 15. [agentic/resilience/wrap.py](#) 16. [agentic/resilience/circuit_breaker.py](#) 17. [agentic/resilience/fallbacks.py](#)

Audio 18. [audio/transcriber.py](#) 19. [audio/tts.py](#) 20. [audio/audio_input.py](#) (VAD) 21. [audio/math_speech.py](#)

Handlers 22. [handlers/ws.py](#) (excerpts) 23. [handlers/audio_pipeline.py](#) 24. [handlers/session_manager.py](#)

Services 25. [services/vision_describe.py](#) 26. [services/nav_dispatcher.py](#) 27. [services/text_turns.py](#) 28. [services/course_slides.py](#)

RAG (ingestion + helpers) 29. [rag/multimodal_rag.py](#) (key functions) 30. [rag/embedding_cache.py](#) 31. [pedagogy/intelligent_ingester.py](#) 32. [pedagogy/course_builder.py](#)

Knowledge Graph (full module) 33. [pedagogy/knowledge_graph/graph.py](#) 34. [pedagogy/knowledge_graph/builder.py](#) 35. [pedagogy/knowledge_graph/concepts_loader.py](#) 36. [pedagogy/knowledge_graph/persistence.py](#) 37. [pedagogy/concept_from_titles.py](#)

Pédagogie (autres) 38. [pedagogy/practice_engine.py](#) 39. [pedagogy/path_recommender.py](#) 40. [pedagogy/skill_tree.py](#) 41. [pedagogy/student_history.py](#) 42. [pedagogy/personalization/engine.py](#) 43. [pedagogy/personalization/learning_style/bayes.py](#) 44. [pedagogy/personalization/bandit/controller.py](#) 45. [pedagogy/personalization/bandit/repo.py](#)

Database / Routes / Observability 46. [database/init_db.py](#) 47. [database/models.py](#) 48. [routes/course.py](#) 49. [routes/student.py](#) 50. [observability/logger.py](#) 51. [observability/analytics.py](#) 52. [observability/kpi_logger.py](#)

1. [agentic/state.py](#)

Rôle : Pydantic **TutorState** partagé entre tous les nœuds des graphes.

```
from pydantic import BaseModel
from typing import Any, Optional

class Action(BaseModel):
    type: str # "navigate", "play_quiz", "show_chunk", ...
    payload: dict[str, Any]

class TutorState(BaseModel):
    # Inputs (set by WS handler before invoke)
    question: str = ""
    raw_question: str = ""
    language: str = "fr"
    course_id: Optional[str] = None
    chapter_idx: Optional[int] = None
    section_idx: Optional[int] = None
    last_slide_content: str = "" # current slide text + vision desc
    history: list[dict] = []
    student_id: str = ""
    session_id: str = ""
    student_level: str = "lycée"
    is_confused: bool = False
    confusion_score: float = 0.0

    # Personalization context
    profile: dict = {}
    knowledge_snapshot: dict = {}
    bandit_decision: dict = {} # {strategy, speech_rate, reasoning, arm_id}
    engagement: dict = {}

    # Set by intent agent
    intent: Optional[Any] = None # IntentDecision

    # Set by rewriter
    rewritten_query: str = ""

    # Set by retriever
    chunks: list[dict] = [] # [{content, idea_id, source, score, ...}]

    # Set by responder
    answer: str = ""
    citations: list[dict] = []
    actions: list[Action] = []
    confidence: float = 0.0
    strategy_used: str = ""

    # Set by reviewer
    grounded: bool = True
    review_feedback: str = ""
    retry_count: int = 0
```

LangGraph compose ces nœuds avec ce state → chaque nœud lit ce dont il a besoin et écrit son résultat.

2. agentic/qa/graph.py

Rôle : compose les 5 nœuds + retry loop + fallback.

```
from langgraph.graph import StateGraph, START, END
from agentic.state import TutorState
from agentic.qa.intent import IntentAgent, RouteAfterIntent
from agentic.qa.rewriter import RewriterAgent
from agentic.qa.retriever import RetrieverAgent
```

```

from agentic.qa.responder import ResponderAgent
from agentic.qa.reviewer import ReviewerAgent, RouteAfterReview, FallbackEmitter
from agentic.resilience.wrap import wrap_with_resilience

```

```

MAX_RESPONDER_RETRIES = 2

```

```

def build_qa_graph(brain, rag) -> StateGraph:

```

```

    """Compile the Q&A LangGraph.

```

```

    7 nodes : intent → [retrieval_decision → rewriter → retriever →]
               responder → qa_review ■ {retry, qa_fallback}

```

```

    """

```

```

    graph = StateGraph(TutorState)

```

```

    # Nodes (chacun wrapped pour résilience : timeout + retry + circuit breaker)

```

```

    graph.add_node("intent", wrap_with_resilience(IntentAgent(brain), "intent"))
    graph.add_node("rewriter", wrap_with_resilience(RewriterAgent(brain), "rewriter"))
    graph.add_node("retriever", wrap_with_resilience(RetrieverAgent(rag), "retriever"))
    graph.add_node("responder", wrap_with_resilience(ResponderAgent(brain), "responder"))
    graph.add_node("qa_review", wrap_with_resilience(ReviewerAgent(brain), "qa_review"))
    graph.add_node("qa_fallback", FallbackEmitter())

```

```

    # Routing

```

```

    graph.add_edge(START, "intent")

```

```

    # After intent : route by intent type

```

```

    def route_after_intent(state: TutorState) -> str:

```

```

        intent_type = getattr(state.intent, "type", "question")

```

```

        if intent_type in ("navigation", "off_topic", "confusion_signal"):

```

```

            return "responder" # skip retrieval pour ces 3 types

```

```

        return "rewriter" # question / clarification → full pipeline

```

```

    graph.add_conditional_edges("intent", route_after_intent, {

```

```

        "rewriter": "rewriter",

```

```

        "responder": "responder",

```

```

    })

```

```

    graph.add_edge("rewriter", "retriever")

```

```

    graph.add_edge("retriever", "responder")

```

```

    graph.add_edge("responder", "qa_review")

```

```

    # After review : grounded? END / retry / fallback

```

```

    def route_after_review(state: TutorState) -> str:

```

```

        if state.grounded:

```

```

            return "end"

```

```

        if state.retry_count < MAX_RESPONDER_RETRIES:

```

```

            state.retry_count += 1

```

```

            return "responder" # retry with feedback

```

```

        return "qa_fallback" # max retries → safe refusal

```

```

    graph.add_conditional_edges("qa_review", route_after_review, {

```

```

        "end": END,

```

```

        "responder": "responder",

```

```

        "qa_fallback": "qa_fallback",

```

```

    })

```

```

    graph.add_edge("qa_fallback", END)

```

```

    log.info("Q&A graph compiled (7 nodes: intent → [retrieval_decision → rewriter → retriever →] responder -

```

```

    return graph.compile()

```

MAX_RESPONDER_RETRIES = 2 → max 3 tentatives totales (1 initial + 2 retries).

3. agentic/qa/intent.py

Rôle : classifier l'intent de la question. SIGHT-first, LLM fallback.

```
class IntentDecision(BaseModel):
    type: str # question | clarification | confusion_signal | navigation | off_topic
    confidence: float
    needs_retrieval: bool = True
    is_definition: bool = False
    definition_term: str = ""
    anchored_concept: str = ""
    needs_rewrite: bool = False
    rewritten: str = ""
    payload: dict = {} # nav_action, nav_target, etc.

class IntentAgent:
    def __init__(self, brain):
        self.brain = brain

    async def __call__(self, state: TutorState) -> dict:
        text = state.question.strip()
        log.info("■ intent USER text : %r", text[:150])

        # 1. SIGHT classifier first (fast + free)
        try:
            from pedagogy.confusion.detector import predict_confusion
            is_confused, prob = predict_confusion(text)
            if is_confused:
                log.info("intent: SIGHT-classified as confusion_signal (prob=%0.2f)", prob)
                state.intent = IntentDecision(
                    type="confusion_signal",
                    confidence=float(prob),
                    needs_retrieval=False,
                )
                state.is_confused = True
                state.confusion_score = float(prob)
                return {"intent": state.intent}
        except Exception as exc:
            log.debug(f"SIGHT skipped: {exc}")

        # 2. LLM intent classifier
        prompt = self._build_intent_prompt(text, state.language)
        from ai.llm_router import get_default_router
        router = get_default_router()
        raw = router.invoke(prompt, prefer="openai", temperature=0.0, max_tokens=300)
        log.info("■ intent LLM raw : %r", raw[:200] if raw else None)

        decision = self._parse_intent(raw, text)
        log.info("intent: %s (conf=%0.2f, retrieve=%s) | rewriter merged: %s (%d→%d, %r)",
            decision.type, decision.confidence, decision.needs_retrieval,
            "yes" if decision.needs_rewrite else "no-op",
            len(text), len(decision.rewritten or text),
            decision.rewritten[:60] if decision.rewritten else "")

        state.intent = decision
        return {"intent": state.intent}
```

Prompt LLM (FR) :

Tu es un classificateur d'intent pour un tuteur IA.
Question : "{text}"

Classifie en :

- "question" : factuel, demande d'info
- "clarification" : reformuler une explication précédente
- "confusion_signal" : "je comprends pas", "c'est confus"
- "navigation" : "slide suivante", "répète"
- "off_topic" : hors-sujet du cours

JSON STRICT :

```
{
  "intent": "question",
```

```

"confidence": 0.95,
"needs_retrieval": true,
"is_definition": true,
"definition_term": "...",
"anchored_concept": "...",
"needs_rewrite": true,
"rewritten": "Qu'est-ce que la RI ?"
}

```

4. agentic/qa/rewriter.py

Rôle : amplifier le rappel RAG en ré-écrivant la query.

```

class RewriterAgent:
    def __init__(self, brain):
        self.brain = brain

    async def __call__(self, state: TutorState) -> dict:
        intent = state.intent
        original = state.question

        # If intent already provided a rewrite, use it
        if intent and getattr(intent, "rewritten", "") and getattr(intent, "needs_rewrite", False):
            state.rewritten_query = intent.rewritten.strip()
            log.info("rewriter: skipped (merged with intent, rewrite=%d chars)", len(state.rewritten_query))
            return {"rewritten_query": state.rewritten_query}

        # Otherwise, decide whether to rewrite
        if len(original) >= 30 and "?" in original:
            # Already a complete question, no rewrite needed
            state.rewritten_query = original
            return {"rewritten_query": original}

        # Short / cryptic queries get rewritten
        prompt = self._build_rewrite_prompt(original, state.language, state.course_id)
        from ai.llm_router import get_default_router
        raw = get_default_router().invoke(prompt, prefer="openai", max_tokens=100)
        rewritten = self._parse_rewrite(raw, fallback=original)
        log.info("■ rewriter ORIGINAL: %r", original[:80])
        log.info("■ rewriter REWRITTEN: %r", rewritten[:80])
        state.rewritten_query = rewritten
        return {"rewritten_query": rewritten}

```

Prompt :

Réécrits cette question d'étudiant pour qu'elle soit explicite et complète,
 en ajoutant le contexte du chapitre actuel ({chapter_title}).
 Question originale : "{original}"
 Réécriture (1 phrase, max 20 mots) :

5. agentic/qa/retriever.py

Rôle : Q&A retriever avec optional KG-augmentation.

```

class RetrieverAgent:
    def __init__(self, rag):
        self.rag = rag

    async def __call__(self, state: TutorState) -> dict:
        if not state.intent.needs_retrieval:

```

```

        return {"chunks": []}

query = state.rewritten_query or state.question
log.info("■ retriever expanded_query: %r", query)

chunks_with_scores = await asyncio.to_thread(
    self.rag.retrieve_chunks,
    query,
    k=Config.RAG_NUM_RESULTS,
    course_id=state.course_id,
    current_chapter_idx=state.chapter_idx,
    strict_chapter=False,          # cross-chapter search OK pour Q&A
)

# Convert to plain dicts
chunks = []
for doc, score, source in (chunks_with_scores or []):
    chunks.append({
        "content": doc.page_content,
        "idea_id": doc.metadata.get("idea_id", ""),
        "label": doc.metadata.get("idea_label", ""),
        "ch": doc.metadata.get("chapter_idx"),
        "sec": doc.metadata.get("section_idx"),
        "source": source,
        "score": float(score) if score is not None else 0.0,
        "metadata": doc.metadata,
    })

# Per-chunk log
for i, c in enumerate(chunks):
    log.info("■ chunk[%d] score=%.3f ch=%s sec=%s seen=%s mastery=%.2f | %r",
            i, c["score"], c["ch"], c["sec"],
            "Y" if c["idea_id"] in state.knowledge_snapshot.get("seen_idea_ids", set()) else "N",
            0.5, # placeholder for mastery lookup
            c["content"][:120])

# KG-augmentation (optional)
kg_augmented = []
if Config.RAG_USE_GRAPH_EXPANSION and chunks:
    kg_augmented = self._augment_with_kg(chunks[0], state)

all_chunks = chunks + kg_augmented
log.info("retriever: %d direct + %d kg-augmented (course=%s ch=%s, seen=%d, review_mode=%s)",
        len(chunks), len(kg_augmented),
        (state.course_id or "")[:16], state.chapter_idx,
        0, False)
state.chunks = all_chunks
return {"chunks": all_chunks}

def _augment_with_kg(self, top1: dict, state: TutorState) -> list[dict]:
    """Add prereqs / examples / illustrated concept of the top-1 chunk."""
    try:
        from deps import get_rag
        from pedagogy.knowledge_graph import get_or_build
        kg = get_or_build(get_rag())
        top1_id = top1.get("idea_id")
        if not top1_id:
            return []

        extras = []
        for prereq in kg.prerequisites_of(top1_id, transitive=False)[:2]:
            extras.append({
                "content": prereq.text,
                "idea_id": prereq.idea_id,
                "label": "prereq",
                "score": 0.0,
                "source": "kg-augmented",
            })
        for example in kg.examples_of(top1_id)[:1]:
            extras.append({
                "content": example.text,

```

```

        "idea_id": example.idea_id,
        "label": "example",
        "score": 0.0,
        "source": "kg-augmented",
    })
    return extras
except Exception:
    return []

```

6. agentic/qa/responder.py

Rôle : génère la réponse pédagogique. ~1 800 lignes — fichier le plus complexe du Q&A graph.

6.1 Structure générale

```

class ResponderAgent:
    def __init__(self, brain):
        self.brain = brain

    async def __call__(self, state: TutorState) -> dict:
        intent = state.intent
        intent_type = intent.type if intent else "question"
        lang = (state.language or "fr")[:2]
        slide = state.last_slide_content or ""

        # Static fast paths (no LLM)
        if intent_type == "navigation":
            return self._handle_navigation(state, lang)
        if intent_type == "off_topic":
            return self._handle_off_topic(state, lang)

        # LLM-backed paths
        # ... (build prompt → call LLM → parse → guardrail → return)

```

6.2 Fast path : navigation

```

def _handle_navigation(self, state: TutorState, lang: str) -> dict:
    nav_action = ""
    nav_target = ""
    if state.intent and isinstance(state.intent.payload, dict):
        nav_action = str(state.intent.payload.get("nav_action", "") or "").strip().lower()
        nav_target = str(state.intent.payload.get("nav_target", "") or "").strip()

    replies = _NAV_REPLIES_BY_ACTION.get(lang, _NAV_REPLIES_BY_ACTION["fr"])
    answer = replies.get(nav_action) or replies["_default"]

    actions = [Action(type="navigate", payload={
        "nav_action": nav_action or "next",
        "nav_target": nav_target,
    })]
    log.info("responder: nav action=%r target=%r → answer=%r", nav_action, nav_target, answer[:80])
    state.answer = answer
    state.actions = actions
    state.grounded = True # nav doesn't need grounding
    return {"answer": answer, "actions": actions, "grounded": True}

```

_NAV_REPLIES_BY_ACTION :

```

_NAV_REPLIES_BY_ACTION = {
    "fr": {

```

```

        "next": "D'accord, je passe à la suite.",
        "previous": "D'accord, je reviens à la slide précédente.",
        "repeat": "D'accord, je répète.",
        "skip": "D'accord, je saute cette section.",
        "go_to_concept": "D'accord, je vais à ce concept.",
        "explain_more": "D'accord, je détaille plus.",
        "slow_down": "D'accord, je ralentis.",
        "_default": "D'accord, je reviens sur ce point.",
    },
    "en": {...}
}

```

6.3 Build prompt — question mode (mode principal)

```

def _build_qa_prompt(self, state, slide, chunks, history, lang) -> str:
    # 1. Personalization prefix (style cognitif + bandit strategy)
    pers_prefix = ""
    try:
        from pedagogy.personalization.engine import PersonalizationEngine
        pers_ctx = state.profile or {}
        pers_prefix = PersonalizationEngine.build_prompt_prefix(pers_ctx, lang=lang)
        log.info("■ personalization | style=%s pace=%s depth=%s tone=%s | avg_rt=%.1fs confusion_rate=%.2f diff",
                pers_ctx.get("style"), pers_ctx.get("pace"), pers_ctx.get("depth"),
                pers_ctx.get("tone"), pers_ctx.get("avg_response_time_s", 0),
                pers_ctx.get("confusion_rate", 0), pers_ctx.get("preferred_explanation_depth"))
    except Exception:
        pass

    # 2. Bandit strategy block
    bandit_block = ""
    if state.bandit_decision and state.bandit_decision.get("reasoning"):
        bandit_block = (
            "<<INSTRUCTION_INTERNE_NE_PAS_MENTIONNER_DANS_LA_REPONSE>>\n"
            f"Stratégie pédagogique à appliquer : {state.bandit_decision['reasoning']}\n"
            "<<FIN_INSTRUCTION_INTERNE>>\n\n"
        )

    # 3. Knowledge snapshot block
    snapshot_block = ""
    if state.knowledge_snapshot:
        try:
            from pedagogy.student_knowledge import to_prompt_context, StudentKnowledgeSnapshot
            snap_obj = StudentKnowledgeSnapshot(**state.knowledge_snapshot)
            snapshot_block = to_prompt_context(snap_obj, lang=lang)
        except Exception:
            pass

    # 4. Tone / instruction (mode-specific)
    if lang == "fr":
        tone = self._get_tone_fr(state)
        course_bound_rule = (
            "■ RÈGLE – TUTEUR LIMITÉ AU MATÉRIEL DU COURS : tu réponds à partir de "
            "TOUT le matériel du cours fourni ci-dessous (la SLIDE EN COURS et le "
            "CONTEXTE INTERNE – qui peut contenir des extraits de N'IMPORTE QUELLE "
            "partie du cours). L'étudiant peut poser des questions qui dépassent la "
            "slide affichée – c'est légitime. Tu n'es PAS un chatbot encyclopédique : "
            "tu n'utilises PAS ta connaissance générale, Wikipédia, ni d'exemples "
            "extérieurs au cours. Si la question porte sur un sujet qui n'est ni dans "
            "la slide ni dans le contexte interne, tu DOIS répondre honnêtement par "
            "UNE SEULE phrase, ex : 'Ce point n'est pas abordé dans ce cours – je ne "
            "peux pas l'expliquer ici sans m'écarter du programme.' (et laisse "
            "supporting_chunks vide). N'invente RIEN, ne paraphrase RIEN depuis "
            "l'extérieur du cours."
        )
        grounding_rule = (
            "ANCRAGE : la SLIDE EN COURS est ta source de vérité prioritaire. "
            "Si elle contient des exemples, des tableaux, des définitions – utilise-les. "
            "Le CONTEXTE INTERNE est un complément ; chaque morceau y est étiqueté par [id:xxx]. "
            "Certains morceaux portent un label supplémentaire issu du graphe pédagogique : "

```

```

        "[id:xxx | prerequisites] (à connaître AVANT pour comprendre la réponse), "
        "[id:xxx | exemple concret] (cas concret du concept), "
        "[id:xxx | concept illustre] (le concept général dont la question est un exemple). "
        "Sers-toi en pour structurer : 'pour comprendre cela, il faut d'abord savoir que ...' "
        "(prereq) ou 'concrètement : ...' (exemple). Si tu utilises un morceau, ajoute son "
        "id dans \"supporting_chunks\". Si la réponse vient uniquement de la slide, laisse "
        "\"supporting_chunks\" vide."
    )

    return (
        f"{pers_prefix}\n"
        f"{bandit_block}"
        f"{snapshot_block}"
        f"Tu es Smart Teacher, un tuteur IA qui répond à la question d'un étudiant en cours.\n\n"
        f"{course_bound_rule}\n\n"
        f"INSTRUCTION PÉDAGOGIQUE (uniquement si la question est couverte par le cours) : {tone}\n\n"
        f"{grounding_rule}\n\n"
        f"Le champ \"answer\" est lu à voix haute : pas de markdown, pas de [id:...]. "
        f"dans le texte parlé – les ids vont uniquement dans \"supporting_chunks\".\n\n"
        f"■■■■ SLIDE EN COURS ■■■■\n{slide[:_SLIDE_CONTENT_CAP]}\n\n"
        f"■■■■ CONTEXTE INTERNE (chaque morceau a un [id:xxx]) ■■■■\n{chunks_block or '(aucun)'}\n\n"
        f"■■■■ ÉCHANGES RÉCENTS (continuité – ne les répète pas) ■■■■\n{history_text}\n\n"
        f"■■■■ QUESTION ACTUELLE ■■■■\n{state.question}\n\n"
        f"Réponds UNIQUEMENT en JSON strict, sans markdown :\n"
        f"{{\"answer\": \"...\", \"supporting_chunks\": [\"id1\", \"id2\"]}}")
    )

```

`_SLIDE_CONTENT_CAP = 4000` chars — limite la slide pour éviter de saturer le contexte LLM.

6.4 Tone par engagement

```

def _get_tone_fr(self, state: TutorState) -> str:
    tone_default = (
        "Réponds comme un VRAI ENSEIGNANT qui explique en 5-7 phrases naturelles, "
        "pas comme un chatbot qui balance un seul exemple. Structure obligatoire :\n"
        " 1. DÉFINITION précise du concept (1-2 phrases, vocabulaire technique correct).\n"
        " 2. REFORMULATION en mots plus simples pour confirmer la compréhension.\n"
        " 3. INTUITION : à quoi ça sert, dans quel contexte on l'utilise (1 phrase).\n"
        " 4. UN EXEMPLE CONCRET tiré du cours, pas de Wikipédia (1-2 phrases).\n"
        " 5. (Optionnel) Lien avec un concept proche ou prérequis si présent dans le cours."
    )

    # Engagement modulation (set upstream by WS handler)
    engagement = state.engagement or {}
    label = engagement.get("label")
    if label == "disengaged":
        log.info("responder: engagement=disengaged → concise+prompt mode")
        return "ALERT: student low-engagement. Keep your answer SHORT (3 phrases max) + finish with one engagin
    if label == "engaged":
        log.debug("responder: engagement=engaged → push-deeper mode")
        # default tone is fine
    return tone_default

```

6.5 Off-topic guardrail

```

def _detect_leak_or_offtopic(self, answer, slide, chunks, lang, has_citations):
    """Returns (should_reject: bool, reason: str)."""
    answer_tokens = self._tokenize(answer.lower())
    if len(answer_tokens) < 3:
        return False, ""

    source_tokens = set()
    source_tokens.update(self._tokenize(slide.lower()))
    for c in chunks:
        source_tokens.update(self._tokenize(c.get("content", "").lower()))

```

```

# Filter stopwords
stopwords = self._stopwords_for(lang)
answer_tokens_filtered = answer_tokens - stopwords
source_tokens_filtered = source_tokens - stopwords

if not answer_tokens_filtered:
    return False, ""

common = answer_tokens_filtered & source_tokens_filtered
overlap = len(common) / len(answer_tokens_filtered)

log.info("■ guardrail | overlap=%.2f (threshold=%.2f) | answer_words=%d source_words=%d | common=%d | has_citations=%d"
        % (overlap, Config.GROUNDING_OVERLAP_THRESHOLD,
            len(answer_tokens_filtered), len(source_tokens_filtered),
            len(common), has_citations))

if overlap < Config.GROUNDING_OVERLAP_THRESHOLD and not has_citations:
    return True, f"low_overlap_no_citations (overlap={overlap:.2%})"

# Detect known meta-instruction leaks
leak_phrases = ["instruction interne", "stratégie pédagogique", "<<instruction_"]
if any(p in answer.lower() for p in leak_phrases):
    return True, "meta_instruction_leak"

return False, ""

```

6.6 _parse_qa_response

```

def _parse_qa_response(self, raw: str, id_to_chunk: dict[str, dict]) -> tuple[str, list[dict]]:
    """Parse LLM JSON output into (answer_text, citations).

    Falls back to (raw_text, []) when JSON is malformed."""
    if not raw:
        return "", []
    data = _extract_json(raw)
    if not data:
        return raw, []

    answer = str(data.get("answer", "")).strip()
    raw_chunks = data.get("supporting_chunks", []) or []

    citations = []
    for c in raw_chunks:
        if isinstance(c, str) and c.startswith("id:"):
            chunk_id = c[3:]
            if chunk_id in id_to_chunk:
                citations.append({"id": chunk_id, **id_to_chunk[chunk_id]})

    return answer, citations

```

6.7 Retry with reviewer feedback

Quand le reviewer rejette, le responder est re-call avec le feedback ajouté :

```

async def __call__(self, state: TutorState) -> dict:
    # ... build prompt ...

    if state.retry_count > 0 and state.review_feedback:
        prompt += (
            f"\n\n■ TENTATIVE PRÉCÉDENTE REJETÉE par l'examineur – raison : {state.review_feedback}\n"
            f"Corrige ce point précis dans ta nouvelle réponse. Si tu ne peux pas répondre "
            f"depuis le matériel du cours, dis-le honnêtement."
        )
        log.info("responder: retry with reviewer feedback (%r)", state.review_feedback[:80])

    # ... call LLM ...

```

7. agentic/qa/reviewer.py

Rôle : Self-RAG reviewer LLM.

```
class ReviewerAgent:
    def __init__(self, brain):
        self.brain = brain

    async def __call__(self, state: TutorState) -> dict:
        if state.intent and state.intent.type in ("navigation", "off_topic"):
            # Skip review for non-LLM paths
            state.grounded = True
            return {"grounded": True, "review_feedback": ""}

        prompt = self._build_review_prompt(state)
        from ai.llm_router import get_default_router
        raw = get_default_router().invoke(prompt, prefer="openai", max_tokens=200)
        log.info("■ reviewer LLM raw : %r", raw[:200] if raw else None)

        verdict = self._parse_verdict(raw)
        log.info("■ reviewer VERDICT | grounded=%s | feedback=%r | answered=%r",
                verdict["grounded"], verdict["feedback"][:120], state.answer[:120])

        if not verdict["grounded"]:
            if state.retry_count < MAX_RESPONDER_RETRIES:
                log.info("qa_review: not grounded → retry responder (##d)", state.retry_count + 1)
            else:
                log.info("qa_review: max retries reached, routing to fallback")

        state.grounded = bool(verdict["grounded"])
        state.review_feedback = verdict["feedback"]
        return {"grounded": state.grounded, "review_feedback": state.review_feedback}

    def _build_review_prompt(self, state) -> str:
        return (
            f"La réponse suivante est-elle GROUNDÉE dans la slide + chunks fournis ?\n"
            f"\\"Groundée\\" = chaque affirmation factuelle vient du contenu (pas inventée, pas Wikipédia).\n\n"
            f"Slide : \n{state.last_slide_content[:2000]}\n\n"
            f"Chunks : \n{self._format_chunks_summary(state.chunks)}\n\n"
            f"Question : {state.question}\n"
            f"Réponse : {state.answer}\n\n"
            f"JSON STRICT : {{\\"grounded\\": true|false, \\"feedback\\": \\"...si pas grounded, dis pourquoi en 1 p"
        )

    def _parse_verdict(self, raw: str) -> dict:
        data = _extract_json(raw) or {}
        return {
            "grounded": bool(data.get("grounded", True)),
            "feedback": str(data.get("feedback", "")).strip(),
        }

class FallbackEmitter:
    """Emit a safe refusal when responder retries are exhausted."""
    async def __call__(self, state: TutorState) -> dict:
        lang = (state.language or "fr")[:2]
        if lang == "fr":
            answer = ("Je ne suis pas sûr de pouvoir répondre précisément à cette question "
                    "à partir du cours. Pouvez-vous reformuler ou préciser ?")
        else:
            answer = ("I'm not sure I can answer this from the course material. "
                    "Can you rephrase or clarify?")
        log.info("qa_fallback: emitting safe refusal (lang=%s)", lang)
        state.answer = answer
```

```

state.citations = []
state.grounded = True # mark as terminal
return {"answer": answer, "citations": [], "grounded": True}

```

8. agentic/teaching/graph.py

Rôle : compose les 6 nœuds du graphe Teaching.

```

def build_teaching_graph(brain, rag) -> StateGraph:
    graph = StateGraph(TutorState)

    graph.add_node("planner", wrap_with_resilience(PlannerAgent(brain), "planner"))
    graph.add_node("context", wrap_with_resilience(ContextAgent(rag), "context"))
    graph.add_node("adaptation", wrap_with_resilience(AdaptationAgent(), "adaptation"))
    graph.add_node("narrator", wrap_with_resilience(NarratorAgent(brain), "narrator"))
    graph.add_node("teaching_review", wrap_with_resilience(TeachingReviewerAgent(brain), "teaching_review"))
    graph.add_node("teaching_fallback", TeachingFallbackEmitter())

    graph.add_edge(START, "planner")
    graph.add_edge("planner", "context")
    graph.add_edge("context", "adaptation")
    graph.add_edge("adaptation", "narrator")
    graph.add_edge("narrator", "teaching_review")

    def route_after_review(state) -> str:
        return "end" if state.grounded else "teaching_fallback"

    graph.add_conditional_edges("teaching_review", route_after_review, {
        "end": "end",
        "teaching_fallback": "teaching_fallback",
    })
    graph.add_edge("teaching_fallback", END)

    log.info("teaching graph compiled (6 nodes: planner→context→adaptation→narrator→review→[fallback])")
    return graph.compile()

```

9. agentic/teaching/planner.py

Rôle : décompose la slide en plan structuré (definition / example / practice / recap).

```

class PlanStep(BaseModel):
    type: str # "definition" | "theorem" | "example" | "practice" | "recap"
    content: str # mini-narration text
    depth: str # "basic" | "balanced" | "deep"
    concept: str # main concept identifier
    chunk_ids: list[str] = [] # supporting RAG chunks

class PlannerAgent:
    def __init__(self, brain):
        self.brain = brain

    async def __call__(self, state: TutorState) -> dict:
        slide = state.last_slide_content or ""
        if not slide:
            return {"plan": []}

        prompt = self._build_plan_prompt(slide, state.language, state.student_level)
        log.info("■ planner PROMPT | slide_chars=%d level=%s lang=%s",
            len(slide), state.student_level, state.language)

```

```

raw = get_default_router().invoke(prompt, prefer="openai", max_tokens=600)
log.info("■ planner LLM RAW (chars=%d): %r", len(raw or ""), (raw or "")[:200])

plan = self._parse_plan(raw)
for i, step in enumerate(plan):
    log.info("■ plan[%d] type=%s depth=%s concept=%s | %r",
            i, step.type, step.depth, step.concept, step.content[:80])
state.plan = plan
return {"plan": plan}

def _build_plan_prompt(self, slide, lang, level) -> str:
    if lang == "fr":
        return (
            f"Tu es un planificateur pédagogique. Décompose cette slide en étapes "
            f"naturelles d'enseignement pour un niveau {level}. Chaque étape doit "
            f"avoir un type, un contenu, et une profondeur.\n\n"
            f"Slide : \n{slide[:2500]}\n\n"
            f"Types possibles : definition | theorem | example | practice | recap\n"
            f"Profondeurs : basic | balanced | deep\n\n"
            f"JSON : \n"
            f"{{\n  \"plan\": [\n"
            f"    {{\n      \"type\": \"definition\", \"content\": \"...\", \"depth\": \"basic\"}},\n"
            f"    {{\n      \"type\": \"example\", \"content\": \"...\", \"depth\": \"concrete\"}},\n"
            f"    {{\n      \"type\": \"recap\", \"content\": \"...\", \"depth\": \"summary\"}}\n"
            f"  ]}}\n"
        )

```

10. agentic/teaching/context.py

Rôle : enrichit chaque étape du plan avec RAG chunks du chapitre courant.

```

class ContextAgent:
    def __init__(self, rag):
        self.rag = rag

    async def __call__(self, state: TutorState) -> dict:
        plan = getattr(state, "plan", []) or []
        if not plan:
            return {}

        for i, step in enumerate(plan):
            # Retrieve 2-3 supporting chunks for this step
            chunks_with_scores = await asyncio.to_thread(
                self.rag.retrieve_chunks,
                step.content,
                k=3,
                course_id=state.course_id,
                current_chapter_idx=state.chapter_idx,
                strict_chapter=True,          # focus sur le chapitre courant
            )
            step.chunk_ids = [doc.metadata.get("idea_id", "") for doc, _, _ in (chunks_with_scores or [])]
            for j, (doc, score, source) in enumerate(chunks_with_scores or []):
                log.info("■ ctx_chunk[%d.%d] score=%.3f | %r",
                        i, j, score, doc.page_content[:100])

        return {"plan": plan}

```

11. agentic/teaching/adaptation.py

Rôle : adapte le plan selon profile + KG snapshot + bandit.

```

class AdaptationAgent:
    async def __call__(self, state: TutorState) -> dict:
        plan = getattr(state, "plan", []) or []
        snap = state.knowledge_snapshot or {}
        profile = state.profile or {}
        bandit = state.bandit_decision or {}

        log.info("■ adaptation level=%s style=%s | strong=%s weak=%s confused=%s | strategy=%s rate=%s",
            profile.get("level"), profile.get("style"),
            snap.get("strong_concepts", []), snap.get("weak_concepts", []),
            snap.get("recently_confused", []),
            bandit.get("strategy"), bandit.get("speech_rate"))

        adapted_plan = []
        for step in plan:
            new_step = step.copy(deep=True) if hasattr(step, "copy") else PlanStep(**dict(step))

            # 1. Mastery-aware adjustments
            concept = step.concept or ""
            if concept in snap.get("strong_concepts", []):
                # Don't redefine – just reference
                if new_step.type == "definition":
                    new_step.depth = "basic"
                    new_step.content = "[Bridge] " + new_step.content
            elif concept in snap.get("weak_concepts", []):
                # Add example
                if new_step.type == "definition":
                    new_step.depth = "deep"
                    # Insert an example step after
                    adapted_plan.append(new_step)
                    adapted_plan.append(PlanStep(
                        type="example", content=f"Exemple concret de {concept}",
                        depth="concrete", concept=concept,
                    ))
                    continue
            elif concept in snap.get("recently_confused", []):
                # Simplify vocabulary
                new_step.depth = "basic"

            # 2. Bandit strategy bumps
            strategy = bandit.get("strategy")
            if strategy == "decomposition" and new_step.type == "definition":
                new_step.depth = "deep"
            elif strategy == "analogy":
                new_step.content += " (en utilisant une analogie)"

            adapted_plan.append(new_step)

        for i, step in enumerate(adapted_plan):
            log.info("■ adapted_plan[%d] type=%s depth=%s concept=%s",
                i, step.type, step.depth, step.concept)

        state.plan = adapted_plan
        return {"plan": adapted_plan}

```

12. agentic/teaching/narrator.py

Rôle : génère le texte final TTS-ready de la narration.

12.1 Cliffhanger detection

```

_CLIFFHANGER_FR = re.compile(
    r"(?:nous (?:allons|verrons) (?:le |la |les |l[''])?(?:défin|voir|examin|étudi|découvr|abord|expliqu)|"

```

```

r"sera (? :défini|vu|examen|étude|abord|expliqu)|"
r"(?:à|a) (? :venir|suivre)|"
r"que nous (? :allons |)d[ée]finir|"
r"dans (? :la |les )?prochain)\b[^\.]*[.!?...]?\\s*$",
flags=re.IGNORECASE,
)

def _is_cliffhanger(text: str, lang: str) -> bool:
    """True if narration ends with a teaser that never pays off."""
    if not text:
        return False
    tail = text.strip()[-140:]
    pattern = _CLIFFHANGER_FR if lang == "fr" else _CLIFFHANGER_EN
    return bool(pattern.search(tail))

```

12.2 Greeting opener stripper

```
_CLAUSE_END = r"(?:[!?\]\s*|,\s+(?=(?-i:[A-ZÀÂÃÄÊËÈÎïÔÖÙÜÝÇ])))"

_GREETING_OPENER_FR = re.compile(
    rf"""\s*
    (?:(bonjour|bienvenue|salut|hello|hi|cher|chère|chers|chères|
      dans\s+(?:le|la|les|cette|ce|cet)\s+cadre|
      aujourd'hui|maintenant|alors|donc|allez|let's\s+go)
    [^.!?,])*{_CLAUSE_END}
    """,
    flags=re.IGNORECASE | re.VERBOSE,
)

def _strip_greeting_opener(text: str, lang: str) -> str:
    """Iteratively strip greeting clauses from the start (max 3)."""
    pattern = _GREETING_OPENER_FR if lang == "fr" else _GREETING_OPENER_EN
    original_len = len(text)
    for _ in range(3): # max 3 clauses (anti runaway)
        m = pattern.match(text)
        if not m:
            break
        text = text[m.end():].lstrip()
        if len(text) < 50:
            return _ORIGINAL_TEXT_BACKUP # fallback if we stripped too much
    if len(text) < original_len:
        log.info("■ stripped greeting (%d chars)", original_len - len(text))
    return text
```

12.3 Parenthetical duplicates stripper

```

_PAREN_DUPLICATE_RE = re.compile(
    r"([A-Za-zÄ-ÿ0-9' '\-]+(?:\s+[A-Za-zÄ-ÿ0-9' '\-]+){0,7})\s*\^\{2,80}\)",
    flags=re.UNICODE,
)

def _strip_parenthetical_duplicates(text: str) -> str:
    """Strip 'X (X)' or 'X (slight variation)' duplicates.

    Common LLM pattern: 'recherche d'information (RI)' or 'RI (RI)'.
    The acronym in parentheses is fine the FIRST time, but the LLM
    sometimes writes 'la RI (RI)' which is just noise."""
    def repl(m):
        before = m.group(1)
        paren = m.group(2)
        if before.lower() == paren.lower():
            return before
        if SequenceMatcher(None, before.lower(), paren.lower()).ratio() > 0.85:

```

```

        return before
    return m.group(0)

new_text = _PAREN_DUPLICATE_RE.sub(repl, text)
if len(new_text) < len(text):
    log.info("■ stripped paren duplicates (%d chars)", len(text) - len(new_text))
return new_text

```

12.4 NarratorAgent.__call__

```

class NarratorAgent:
    def __init__(self, brain):
        self.brain = brain

    async def __call__(self, state: TutorState) -> dict:
        plan = getattr(state, "plan", []) or []
        slide = state.last_slide_content or ""
        lang = (state.language or "fr")[:2]

        # 1. Build augmented prompt
        prompt = self._build_narrator_prompt(state, plan, slide, lang)
        log.info("■ narrator AUGMENTED PROMPT (chars=%d turns=%d)", len(prompt), len(state.history))

        # 2. Call LLM (Brain.present uses OpenAI/Groq → Ollama fallback)
        narration = await asyncio.to_thread(
            self.brain.present,
            slide_content=slide,
            language=state.language,
            chapter_title=state.profile.get("chapter_title", ""),
            student_level=state.student_level,
            domain=state.profile.get("domain"),
            previous_concept=state.profile.get("last_concept_explained", ""),
            previous_narration_summary=state.profile.get("last_narration_summary", ""),
        )
        log.info("■ narrator LLM RAW (%d chars)", len(narration))

        # 3. Post-processeurs
        narration = _strip_greeting_opener(narration, lang)
        narration = _strip_parenthetical_duplicates(narration)
        if _is_cliffhanger(narration, lang):
            log.warning("narrator: cliffhanger detected – bumping max_tokens next time")
        narration = self.brain._clean_for_speech(narration, lang)

        state.answer = narration
        return {"answer": narration}

```

13. agentic/teaching/reviewer.py

Rôle : valide la narration (grounding sur slide + plan).

```

class TeachingReviewerAgent:
    def __init__(self, brain):
        self.brain = brain

    async def __call__(self, state: TutorState) -> dict:
        narration = state.answer or ""
        slide = state.last_slide_content or ""
        plan = getattr(state, "plan", []) or []

        if len(narration) < 50:
            log.warning("teaching reviewer: narration too short (%d chars) → fallback", len(narration))
            state.grounded = False
            return {"grounded": False}

```

```

# Quick grounding check : tous les concepts du plan sont-ils mentionnés ?
plan_concepts = {step.concept for step in plan if step.concept}
narration_lower = narration.lower()
missing = [c for c in plan_concepts if c.lower() not in narration_lower]
if len(missing) > len(plan_concepts) // 2:
    log.warning("teaching reviewer: %d/%d plan concepts missing from narration",
                len(missing), len(plan_concepts))
    state.grounded = False
    return {"grounded": False}

log.info("■ teaching reviewer VERDICT | grounded=True | plan_concepts_covered=%d/%d",
        len(plan_concepts) - len(missing), len(plan_concepts))
state.grounded = True
return {"grounded": True}

```

14. agentic/teaching/fallback.py

Rôle : narration générique de secours.

```

class TeachingFallbackEmitter:
    async def __call__(self, state: TutorState) -> dict:
        slide = state.last_slide_content or ""
        lang = (state.language or "fr")[:2]
        plan = getattr(state, "plan", []) or []
        main_concept = plan[0].concept if plan else ""

        if lang == "fr":
            answer = (
                f"Cette slide aborde le concept de {main_concept or 'la recherche d''information'}. "
                f"Je vais vous laisser la lire et n'hésitez pas si vous avez des questions."
            )
        else:
            answer = (
                f"This slide covers {main_concept or 'the topic'}. "
                f"Take a moment to read it, and let me know if you have questions."
            )
        log.info("teaching_fallback: emitting safe narration (lang=%s)", lang)
        state.answer = answer
        state.grounded = True
        return {"answer": answer, "grounded": True}

```

15. agentic/resilience/wrap.py

Rôle : décorateur qui ajoute timeout + retry + circuit breaker à un nœud LangGraph.

```

def wrap_with_resilience(node, name: str, timeout_s: float = None, max_retries: int = 1):
    """Wrap a LangGraph node with :
    - timeout (asyncio.wait_for)
    - retry on transient exceptions
    - circuit breaker (skip after N consecutive failures)
    - latency logging
    """
    breaker = CircuitBreaker(name, fail_threshold=5, reset_after_s=300)

    async def wrapped(state: TutorState) -> dict:
        if breaker.is_open():
            log.warning("resilience: circuit OPEN for node=%s → skip", name)
            return {}

        for attempt in range(max_retries + 1):

```

```

t0 = time.time()
try:
    if timeout_s:
        result = await asyncio.wait_for(node(state), timeout=timeout_s)
    else:
        result = await node(state)
    breaker.record_success()
    latency = time.time() - t0
    log.info("resilience event: node=%s kind=ok latency=%.3f", name, latency)
    return result
except asyncio.TimeoutError:
    log.warning("resilience: node=%s timeout (%.1fs) attempt %d", name, timeout_s, attempt)
    breaker.record_failure()
except Exception as exc:
    log.warning("resilience: node=%s exception attempt %d: %s", name, attempt, exc)
    breaker.record_failure()
    if attempt == max_retries:
        raise

return {} # all retries failed

return wrapped

```

16. agentic/resilience/circuit_breaker.py

Rôle : bloque les calls vers un node qui fail répétitivement.

```

class CircuitBreaker:
    def __init__(self, name: str, fail_threshold: int = 5, reset_after_s: float = 300):
        self.name = name
        self.fail_threshold = fail_threshold
        self.reset_after_s = reset_after_s
        self.failures = 0
        self.opened_at = 0.0

    def is_open(self) -> bool:
        if self.failures < self.fail_threshold:
            return False
        if time.time() - self.opened_at >= self.reset_after_s:
            self.failures = 0
            self.opened_at = 0.0
            log.info("■ circuit_breaker RESET | service=%s (reset_after_s=%.0f elapsed)",
                    self.name, self.reset_after_s)
            return False
        return True

    def record_success(self):
        self.failures = 0
        self.opened_at = 0.0

    def record_failure(self):
        self.failures += 1
        if self.failures == self.fail_threshold:
            self.opened_at = time.time()
            log.warning("■ circuit_breaker OPEN | service=%s failures=%d/%d",
                    self.name, self.failures, self.fail_threshold)

```

17. agentic/resilience/fallbacks.py

Fallback responses générique par node (si circuit OPEN) :

```

DEFAULT_FALLBACKS = {
    "intent": {"intent": IntentDecision(type="question", confidence=0.5)},
    "rewriter": {"rewritten_query": ""},
    "retriever": {"chunks": []},
    "responder": {"answer": "Je n'arrive pas à répondre maintenant.", "citations": []},
    "qa_review": {"grounded": True, "review_feedback": ""},
    "narrator": {"answer": "Cette slide aborde un nouveau concept."},
}

```

18. audio/transcriber.py

Rôle : Whisper STT wrapper (faster-whisper backend).

```

class Transcriber:
    def __init__(self):
        self.backend = (Config.STT_BACKEND or "faster-whisper").strip().lower()

        if self.backend != "whisperlivekit":
            log.info("■ WHISPER LOAD START | model_size=%s device=%s compute_type=%s",
                    Config.WHISPER_MODEL_SIZE, Config.WHISPER_DEVICE, Config.WHISPER_COMPUTE)
            self.model = WhisperModel(
                Config.WHISPER_MODEL_SIZE,
                device=Config.WHISPER_DEVICE,
                compute_type=Config.WHISPER_COMPUTE,
                cpu_threads=Config.WHISPER_THREADS,
            )
            log.info("■ WHISPER LOAD DONE | model=%s | took=%.2fs",
                    Config.WHISPER_MODEL_SIZE, time.time() - _wload_t0)

    def transcribe(self, audio: np.ndarray, force_language: str = None) -> tuple[str, float, str, float, float]:
        """Returns (text, stt_time, language, lang_prob, audio_duration)."""
        start = time.time()
        log.info("■ STT START: len=%d samples, force_lang=%s", len(audio), force_language)

        # 1. Validate
        is_valid, reason = self.validate_audio_quality(audio)
        if not is_valid:
            return "", 0.0, "silence", 0.0, len(audio) / Config.SAMPLE_RATE

        # 2. Trim silence
        audio = self.trim_silence(audio)
        audio_duration = len(audio) / Config.SAMPLE_RATE

        # 3. Min duration check
        if audio_duration < Config.STT_MIN_AUDIO_SEC:
            return "", 0.0, "unknown", 0.0, audio_duration

        # 4. Transcribe (faster-whisper)
        segments, info = self.model.transcribe(
            audio,
            language=force_language,
            beam_size=Config.STT_BEAM_SIZE,
            best_of=1,
            temperature=0.0,          # déterministe
            vad_filter=False,          # désactiver VAD agressif
            condition_on_previous_text=False,  # pas de mémoire → moins d'hallucinations
            without_timestamps=True,
            word_timestamps=False,
        )

        # 5. CRITICAL: segments est un GENERATOR, convert immédiatement
        segments = list(segments)
        text = " ".join(s.text.strip() for s in segments).strip()

        lang = getattr(info, "language", "unknown")

```

```

lang_prob = getattr(info, "language_probability", 0.0)
stt_time = time.time() - start

# 6. Reject corrupted Unicode (>30% invalid chars)
if len(text) > 5:
    invalid_categories = {'Cc', 'Cf', 'Cn', 'Co'}
    invalid_count = sum(1 for c in text if unicodedata.category(c) in invalid_categories)
    if invalid_count / len(text) > 0.3:
        log.warning("■ STT Corruption: %d/%d invalid Unicode chars, rejecting",
                    invalid_count, len(text))
        return "", stt_time, lang, lang_prob, audio_duration

rtf = stt_time / audio_duration if audio_duration > 0 else 0
log.info("STT | %r | lang=%s(%.0f%%) | dur=%.2fs | stt=%.2fs | RTF=%.2fx",
        text[:60] + "...", lang, lang_prob*100, audio_duration, stt_time, rtf)

return text, stt_time, lang, lang_prob, audio_duration

```

18.1 trim_silence (Silero VAD)

```

def trim_silence(self, audio: np.ndarray) -> np.ndarray:
    """Strip leading + trailing silence using Silero VAD."""
    if not hasattr(self, "_vad") or self._vad is None:
        return audio # no VAD → return as-is

    speech_timestamps = self._vad(audio, sampling_rate=Config.SAMPLE_RATE)
    if not speech_timestamps:
        return audio[:0] # all silence

    start = speech_timestamps[0]["start"]
    end = speech_timestamps[-1]["end"]
    return audio[start:end]

```

18.2 extract_prosody

```

def extract_prosody(self, text: str, audio_duration: float) -> dict:
    """Extract speech_rate (wpm), hesitation count, markers."""
    words = text.split()
    n_words = len(words)
    speech_rate = (n_words / audio_duration * 60) if audio_duration > 0 else 0

    # Detect hesitations
    HESITATIONS_FR = {"euh", "hum", "ben", "eh", "uhm", "ah"}
    HESITATIONS_EN = {"um", "uh", "uhm", "er", "hmm", "ah"}
    hesitations_fr = sum(1 for w in words if w.lower().strip(",.!?" ) in HESITATIONS_FR)
    hesitations_en = sum(1 for w in words if w.lower().strip(",.!?" ) in HESITATIONS_EN)
    hesitation_count = max(hesitations_fr, hesitations_en)

    markers = []
    if speech_rate < 100 and n_words >= 3:
        markers.append("slow_speech_rate")
    if hesitation_count >= 2:
        markers.append("frequent_hesitations")

    return {
        "speech_rate": int(round(speech_rate)),
        "hesitation_count": hesitation_count,
        "markers": markers,
    }

```

19. audio/tts.py

Rôle : TTS Edge-TTS / ElevenLabs.

```
EDGE_VOICES = {
    "fr": {"female": "fr-FR-DeniseNeural", "male": "fr-FR-HenriNeural"},
    "en": {"female": "en-US-JennyNeural", "male": "en-US-GuyNeural"},
}
DEFAULT_VOICE = "en-US-JennyNeural"
ELEVENLABS_VOICES = {
    "fr": "pNInz6obpgDQGcFmaJgB",      # Adam (French)
    "en": "21m00Tcm4TlvDq8ikWAM",      # Rachel (English)
}

class VoiceEngine:
    def __init__(self):
        self.provider = Config.TTS_PROVIDER          # "edge" | "elevenlabs"
        self.gender = "female"
        self.voice_name = DEFAULT_VOICE
        self.voice_id = DEFAULT_VOICE
        self._el_client = None
        if self.provider == "elevenlabs":
            self._init_elevenlabs()
        log.info("■ TTS provider initialized: %s", self.provider)

    async def generate_audio_async(self, text: str, language_code: str = None,
                                   rate: str = "+0%") -> tuple[bytes, float, str, str, str]:
        """Returns (audio_bytes, duration_s, engine_name, voice_name, mime_type)."""
        if not text or len(text.strip()) < 2:
            return None, 0.0, "none", "none", None

        safe_rate = self._sanitize_rate(rate)

        # Provider chain : ElevenLabs → Edge-TTS fallback
        if self.provider == "elevenlabs" and self._el_client:
            result = await self._synthesize_elevenlabs(text, language_code, safe_rate)
            if result[0] is not None:
                return result
            log.warning("ElevenLabs synthesis failed – fallback Edge-TTS")

        return await self._synthesize_edge(text, language_code, safe_rate)

    async def _synthesize_edge(self, text, language_code, rate) -> tuple[bytes, float, str, str, str]:
        voice = self._pick_edge_voice(language_code)
        start_time = time.time()
        try:
            communicate = edge_tts.Communicate(text=text, voice=voice, rate=rate)
            audio_bytes = b""
            async for chunk in communicate.stream():
                if chunk["type"] == "audio":
                    audio_bytes += chunk["data"]
            duration = time.time() - start_time
            log.info("■ Edge-TTS: %.2fs | voice=%s | rate=%s | %d bytes",
                    duration, voice, rate, len(audio_bytes))
            # Return "edge" (not "edge_tts") so engine name matches get_cache_signatures()
            return audio_bytes, duration, "edge", voice, "audio/mpeg"
        except Exception as exc:
            log.error("■ Edge-TTS synthesis failed: %s", exc)
            return None, 0.0, "none", "none", None

    def get_cache_signatures(self, language_code: str = None) -> list[tuple[str, str]]:
        """Return cache signatures for the active provider (used for cache key generation)."""
        if self.provider == "elevenlabs" and self._el_client:
            lang = (language_code or "en")[:2].lower()
            return [
                ("elevenlabs", ELEVENLABS_VOICES.get(lang, ELEVENLABS_VOICES["en"])),
                ("edge", self._pick_edge_voice(language_code)),
            ]
```

```
return [("edge", self._pick_edge_voice(language_code))]
```

Le retour **"edge"** (pas **"edge_tts"**) est essentiel — sinon le cache TTS écrirait sous deux noms différents (cf. fix anti-doublon).

20. audio/audio_input.py (VAD)

Rôle : Silero VAD wrapper.

```
class VADProcessor:
    def __init__(self):
        log.info("Chargement du modèle VAD Silero...")
        try:
            import torch
            self.model, _utils = torch.hub.load(
                repo_or_dir='snakers4/silero-vad',
                model='silero_vad',
                force_reload=False,
                onnx=False,
            )
            self.get_speech_timestamps = _utils[0]
            self.model.eval()
            log.info("■ VAD Silero chargé")
        except Exception as exc:
            log.error("■ VAD load failed: %s", exc)
            self.model = None

    def detect_speech(self, audio: np.ndarray, threshold: float = None) -> list[dict]:
        """Returns list of {start, end} timestamps for speech segments."""
        if self.model is None:
            return [{"start": 0, "end": len(audio)}]
        threshold = threshold or Config.SPEECH_THRESHOLD
        return self.get_speech_timestamps(
            torch.from_numpy(audio),
            self.model,
            threshold=threshold,
            sampling_rate=Config.SAMPLE_RATE,
        )
```

Silero VAD = ML model léger (~ 3MB) qui détecte si un chunk audio contient de la voix humaine. Sortie : liste d'intervalles **{start, end}** (en samples).

21. audio/math_speech.py

Rôle : post-processeur déterministe qui convertit les notations math en texte parlé.

```
# FR
_MATH_REPLACE_FR = {
    "=": " égale ",
    "≠": " différent de ",
    "≈": " environ ",
    "→": " donne ",
    "⇒": " implique ",
    "⇔": " équivalent à ",
    "+": " plus ",
    "-": " moins ",
    "×": " fois ",
    "÷": " divisé par ",
```

```

"/": " sur ",
"<": " inférieur à ",
">": " supérieur à ",
"≤": " inférieur ou égal à ",
"≥": " supérieur ou égal à ",
"²": " au carré ",
"³": " au cube ",
"∞": " infini ",
"Σ": " somme ",
"Π": " produit ",
"√": " racine de ",
"π": " pi ",
"α": " alpha ", "β": " bêta ", "γ": " gamma ", "δ": " delta ",
"θ": " thêta ", "λ": " lambda ", "μ": " mu ", "σ": " sigma ",
"Σ": " somme ", "Δ": " delta ",
}

def to_speech_friendly(text: str, language: str = "fr") -> str:
    """Convert math notation to plain spoken text."""
    replace = _MATH_REPLACE_FR if language[:2] == "fr" else _MATH_REPLACE_EN
    for symbol, spoken in replace.items():
        text = text.replace(symbol, spoken)

    # Strip LaTeX commands
    text = re.sub(r"\\[a-zA-Z]+\{([^\}]*)\}", r"\1", text)
    text = re.sub(r"\$+([^\$]+)\$", r"\1", text)
    text = re.sub(r"^\^[0-9]+)", r" exposant \1 ", text)
    text = re.sub(r"_([0-9]+)", r" indice \1 ", text)

    # Clean multiple spaces
    text = re.sub(r"\s+", " ", text).strip()
    return text

```

22. handlers/ws.py (excerpts)

~3 800 lignes — le plus gros fichier. Je couvre les sections critiques.

22.1 WebSocket entry point

```

@app.websocket("/ws/{session_id}")
async def websocket_endpoint(websocket: WebSocket, session_id: str):
    # 1. Auth check (JWT from cookie or header)
    token = websocket.cookies.get("smart_teacher_token", "")
    if not token:
        ah = websocket.headers.get("authorization", "")
        if ah.startswith("Bearer "):
            token = ah[7:]
    if not token:
        await websocket.close(code=4401, reason="No token")
        return

    try:
        claims = decode_access_token(token)
        student_id = claims["sub"]
    except Exception:
        await websocket.close(code=4401, reason="Invalid token")
        return

    await websocket.accept()
    log.info("■ WebSocket connecté : %s", session_id[:8])

    # 2. State init

```

```

ctx: Optional[SessionContext] = None
current_presentation_text: str = ""
current_presentation_cursor: int = 0
current_presentation_key: tuple = None
pause_progress_task: Optional[asyncio.Task] = None
pause_started_at: float = 0.0
# ... (~70 nonlocal variables)

# 3. Helper functions (closures over the state above)
async def send(payload: dict):
    if websocket_closed:
        return
    async with send_lock:
        try:
            await websocket.send_json(payload)
        except Exception:
            pass

async def record_pause_point(reason: str, notice_prefix: str = "...") -> str:
    # Uses pause_session, save_presentation_snapshot, start_pause_progress_ticker
    ...

async def _pause_progress_ticker(start_ts: float, slide_id: str, reason: str):
    intervals = [15, 30, 60, 90, 120, 180, 240, 300, 420, 600, 900, 1200, 1800, 2400, 3000, 3600]
    TAIL_STEP_S = 1800
    idx = 0
    try:
        while True:
            next_at = intervals[idx] if idx < len(intervals) else intervals[-1] + TAIL_STEP_S * (idx - len(
            idx += 1
            remaining = (start_ts + next_at) - time.time()
            if remaining > 0:
                await asyncio.sleep(remaining)
            else:
                elapsed_now = time.time() - start_ts
                while idx < len(intervals) and intervals[idx] <= elapsed_now:
                    idx += 1
                elapsed = time.time() - start_ts
                m, s = divmod(int(elapsed), 60)
                bucket = "QUICK" if elapsed < 10 else "NORMAL" if elapsed < 60 else "GAP" if elapsed < 180 else "LONG"
                log.info("[%s] ■ WAITING tick | elapsed=%0.0fs (%dm%02ds) | bucket=%s | reason=%s | slide=%s",
                    session_id[:8], elapsed, m, s, bucket, reason, slide_id or "?")
    except asyncio.CancelledError:
        elapsed = time.time() - start_ts
        m, s = divmod(int(elapsed), 60)
        log.info("[%s] ■ WAITING ticker stopped | total_wait=%0.1fs (%dm%02ds) | reason=%s",
            session_id[:8], elapsed, m, s, reason)
        raise

# 4. Main loop : receive WS messages and dispatch
try:
    while True:
        msg = await websocket.receive_json()
        msg_type = msg.get("type")

        if msg_type == "start_session":
            # Create SessionContext
            ...
        elif msg_type == "present_section":
            # Decide cache reuse, run Teaching graph or replay, stream TTS
            ...
        elif msg_type == "interrupt":
            # Pause flow : audio_progress override, record_pause_point, ticker start
            ...
        elif msg_type == "text_question":
            # Q&A graph invoke
            ...
        elif msg_type == "audio":
            # STT + audio_pipeline
            ...
        # ... (autres handlers)

```

```

except WebSocketDisconnect:
    websocket_closed = True
    log.info("■ WebSocket déconnecté : %s", session_id[:8])
finally:
    # Cleanup : cancel tasks, persist final state
    await cancel_next_slide_prefetch()
    await cancel_presentation_task(notify_client=False)
    await cancel_audio_stream(notify_client=False)
    await cancel_text_question_task()
    if pause_progress_task is not None and not pause_progress_task.done():
        await stop_pause_progress_ticker()

```

22.2 Interrupt handler (pause flow détaillé)

```

elif msg_type == "interrupt":
    interrupt_reason = str(msg.get("reason") or "pause").strip().lower() or "pause"
    turn_id = int(msg.get("turn_id") or 0)
    _kpi = KPITracker.get()
    _kpi.mark_interrupt_detected(session_id, turn_id)

    log.info("[%s] ■ interrupt msg | audio_progress=%r reason=%r current_cursor=%d current_text_len=%d",
            session_id[:8], msg.get('audio_progress'), interrupt_reason,
            current_presentation_cursor, len(current_presentation_text or ""))

    # 1. Audio-progress cursor override
    _audio_progress = msg.get("audio_progress")
    if _audio_progress is not None:
        _override = _cursor_from_audio_progress(_audio_progress, current_presentation_text or "")
        if _override is not None:
            log.info("[%s] ■■ Interrupt cursor override %d → %d (audio_progress=%.2f, narration_len=%d)",
                    session_id[:8], current_presentation_cursor, _override,
                    float(_audio_progress), len(current_presentation_text or ""))
            current_presentation_cursor = _override

    # 2. Navigation reset
    if interrupt_reason in ("navigation_next", "navigation_prev", "repeat"):
        log.info("[%s] ■ navigation interrupt (%s) – resetting cursor 0", session_id[:8], interrupt_reason)
        current_presentation_cursor = 0

    # 3. Save pause point (Redis snapshot + ctx.paused_state)
    await record_pause_point(interrupt_reason, notice_prefix="■ Point d'arrêt mémorisé")

    # 4. Cancel TTS + audio stream
    await cancel_presentation_task(notify_client=True)
    audio_buffer.clear()
    await cancel_audio_stream(notify_client=True, turn_id=turn_id)

    # 5. KPI #1 – interrupt latency
    _interrupt_lat = _kpi.mark_tts_stopped(session_id, turn_id)
    if _interrupt_lat is not None:
        log.info("[%s] ■ interrupt latency = %.0fms", session_id[:8], _interrupt_lat * 1000)

    # 6. Track interaction
    interactions_on_current_slide += 1
    if interrupt_reason == "question":
        questions_in_session += 1
    await cancel_text_question_task()

    # 7. Transition state
    if ctx and interrupt_reason != "pause":
        await dialogue.transition(ctx.session_id, DialogState.LISTENING)
    elif ctx:
        await send_state(DialogState.WAITING, turn_id=turn_id)

    log.info("[%s] ■ Interruption", session_id[:8])

```

23. handlers/audio_pipeline.py

Rôle : pipeline audio mic → STT → confusion fusion → Q&A graph → TTS.

```
async def run_pipeline_streaming(audio_data, session_id, history, *,
                                on_text_chunk=None, on_transcription=None,
                                on_audio_chunk=None, on_state_change=None,
                                force_language=None, course_id=None, ctx=None,
                                transcriber=None, rag=None, voice=None,
                                brain=None, dialogue=None, csv_logger=None,
                                stt_logger=None):

    total_start = time.time()
    utt_id = str(uuid.uuid4())[0:8]

    # ■■ 1. STT ■■
    audio_samples = int(getattr(audio_data, "size", 0) or 0)
    audio_dur_input = audio_samples / 16000.0 if audio_samples else 0.0
    log.info("[%s] ■ STT START | utt=%s samples=%d input_dur=%.2fs force_lang=%s",
            session_id[0:8], utt_id, audio_samples, audio_dur_input, force_language or 'auto')
    stt_t0 = time.time()
    text, stt_time, lang, lang_prob, audio_duration = await asyncio.to_thread(
        transcriber.transcribe, audio_data, force_language)
    log.info("[%s] ■ STT DONE | took=%.2fs | lang=%s(%.0f%%) audio_dur=%.2fs | text=%r",
            session_id[0:8], time.time()-stt_t0, lang, lang_prob*100, audio_duration,
            (text or '')[0:80])

    if not text or len(text.strip()) <= 2:
        log.info("[%s] ■ STT NO-SPEECH | text=%r → abort", session_id[0:8], text)
        return {"no_speech": True}

    if on_transcription:
        await on_transcription(text, lang, round(lang_prob, 2))

    # ■■ 2. Prosody extraction ■■
    prosody = transcriber.extract_prosody(text, audio_duration)
    log.info("[%s] ■■ Prosody: speech_rate=%d wpm, hesitations=%d, confusion_signals=%s",
            session_id[0:8], prosody['speech_rate'], prosody['hesitation_count'], prosody['markers'])

    # ■■ 3. RAG retrieval ■■
    chunks_with_scores = await asyncio.to_thread(
        rag.retrieve_chunks, text, k=Config.RAG_NUM_RESULTS, course_id=course_id)

    # ■■ 4. Confusion detection (SIGHT + prosody fusion) ■■
    is_confused, confusion_reason, q_hash, confusion_count = await dialogue.detect_and_track_confusion(
        session_id=session_id, question_text=text, language=lang,
        history=history, brain=brain, prosody=prosody)

    # ■■ 5. LLM streaming + TTS in parallel ■■
    full_response = ""
    if is_confused:
        # Special : reformulation prompt + preamble audio
        preambles = {"fr": "Permettez-moi de réexpliquer autrement. ", "en": "Let me explain..."}
        preamble = preambles.get(lang[0:2], preambles["fr"])
        await on_text_chunk(preamble, preamble)
        # Synthétise + envoie le préambule en parallèle
        try:
            pre_audio, _, pre_engine, pre_voice, pre_mime = await voice.generate_audio_async(preamble, language)
            if pre_audio and on_audio_chunk:
                await on_audio_chunk(pre_audio, pre_mime)
        except Exception as pre_exc:
            log.warning("Preamble TTS failed: %s", pre_exc)
            confusion_prompt = dialogue.build_confusion_prompt(
                original_question=text, language=lang, last_slide_content=ctx.last_slide_explained if ctx else "",
            )
            question_for_llm = confusion_prompt
        else:
            question_for_llm = text

    # Direct LLM call (Brain.ask)
```

```

full_response, _ = await asyncio.to_thread(
    brain.ask, question_for_llm, reply_language=lang, session_id=session_id)
full_response = brain._clean_for_speech(full_response)

# TTS de la réponse complète
if on_text_chunk:
    await on_text_chunk(full_response, full_response)
audio_bytes, _, tts_engine, tts_voice, mime = await voice.generate_audio_async(
    full_response, language_code=lang)
if audio_bytes and on_audio_chunk:
    await on_audio_chunk(audio_bytes, mime)
    log.info("[%s] ■ Audio streamed: %d bytes", session_id[:8], len(audio_bytes))

# ■■ 6. Logging metrics ■■
total_time = time.time() - total_start
csv_logger.log_turn(audio_duration_sec=audio_duration, stt_time=stt_time,
                    llm_time=time.time()-llm_start, tts_time=0,
                    total_time=total_time, language=lang, ...)
log.info("[%s] ■ STREAMING | STT=%.2fs LLM=... TOTAL=%.2fs",
        session_id[:8], stt_time, total_time)

return {
    "transcription": {"text": text, "language": lang, "confidence": round(lang_prob, 2)},
    "answer": full_response,
    "confusion": {"detected": bool(is_confused), "reason": confusion_reason, ...},
    "performance": {"stt_time": ..., "llm_time": ..., "total_time": ..., "rtf": ...},
}

```

24. handlers/session_manager.py

Rôle : helpers pour `start_session` (init SessionContext + load profile + bandit + course context).

```

async def setup_new_session(session_id, course_id, language, level, student_id, dialogue, rag):
    """Initialise une nouvelle session étudiant."""
    # 1. Create SessionContext in Redis
    ctx = await dialogue.create_session(language=language, level=level, course_id=course_id)
    ctx.student_id = student_id

    # 2. Load student profile
    from pedagogy.personalization.profile import get_or_create_profile
    profile = await get_or_create_profile(session_id, course_id=course_id)

    # 3. Insert learning_session row in Postgres
    from database.crud import create_learning_session
    learning_session_id = await create_learning_session(
        session_id=session_id, student_id=student_id, course_id=course_id, language=language)
    log.info("Created learning session: %s for student %s",
            learning_session_id, student_id)

    # 4. Detect course settings (language + level from DB, override from session arg)
    course_settings = await load_course_settings(course_id)
    log.info("[%s] ■ course settings | language=%s level=%s (from courses.id=%s)",
            session_id[:8], course_settings.language, course_settings.level, course_id[:8])

    # 5. Course analysis (lang detect, topic extract, ...)
    from pedagogy.course_analyzer import analyze_course
    analysis = await analyze_course(course_id, rag)
    ctx.course_analysis = analysis
    log.info("■ Course analyzed: lang=%s level=%s topics=%d chapters=%d",
            analysis['language'], analysis['level'], len(analysis['topics']), analysis['n_chapters'])

    # 6. Transition IDLE → LISTENING
    await dialogue.transition(session_id, DialogState.LISTENING)
    log.info("[%s] ■ Session démarrée | lang=%s level=%s state=LISTENING",
            session_id[:8], language, level)

```

```

return ctx, profile, learning_session_id

def detect_subject(text: str) -> str:
    """Heuristique pour détecter le sujet d'une question (pour analytics)."""
    text_lower = text.lower()
    keywords = {
        "math": ["math", "équation", "calcul", "intégrale", "dérivée"],
        "code": ["python", "javascript", "function", "code", "bug"],
        "science": ["chimie", "physique", "biologie", "atome"],
        "language": ["grammaire", "verbe", "syntaxe", "traduction"],
        # ...
    }
    for subj, kw in keywords.items():
        if any(k in text_lower for k in kw):
            return subj
    return "general"

```

25. services/vision_describe.py

Rôle : description LLM-vision des slides PNG (gpt-4o-mini-vision OU Ollama llava).

```

_CACHE_DIR = Path(Config.LOGS_DIR).parent / "cache" / "vision_descriptions"
_CACHE_DIR.mkdir(parents=True, exist_ok=True)

async def describe_slide_image(image_path: str, lang: str = "fr") -> Optional[dict]:
    """Returns {description, language, main_concept, captured_at, model} ou None."""
    if not image_path or not Config.VISION_DESCRIBE_ENABLED:
        return None

    try:
        with open(image_path, "rb") as f:
            image_bytes = f.read()
            image_md5 = hashlib.md5(image_bytes).hexdigest()
    except Exception as exc:
        log.warning("vision: cannot read image: %s", exc)
        return None

    # 1. Disk cache check
    cache_path = _CACHE_DIR / f"{image_md5}.json"
    if cache_path.exists():
        try:
            data = json.loads(cache_path.read_text(encoding="utf-8"))
            log.info("■ Vision cache HIT (md5=%s)", image_md5[:8])
            return data
        except Exception:
            pass

    # 2. Try OpenAI vision (premium, fast, accurate)
    if not Config.DISABLE_OPENAI:
        result = await _describe_via_openai(image_path, image_bytes, lang)
        if result:
            cache_path.write_text(json.dumps(result, ensure_ascii=False), encoding="utf-8")
            return result

    # 3. Fallback : Ollama llava
    if Config.OLLAMA_VISION_MODEL:
        result = await _describe_via_ollama(image_path, image_bytes, lang)
        if result:
            cache_path.write_text(json.dumps(result, ensure_ascii=False), encoding="utf-8")
            return result

    return None

async def _describe_via_openai(image_path, image_bytes, lang) -> Optional[dict]:

```

```

api_key = os.getenv("OPENAI_API_KEY") or getattr(Config, "OPENAI_API_KEY", "")
if not api_key:
    return None
if getattr(Config, "DISABLE_OPENAI", False):
    log.debug("vision describe : DISABLE_OPENAI=true → skip OpenAI provider")
    return None

try:
    from openai import OpenAI
    client = OpenAI(api_key=api_key)

    # Encode as base64 data URL
    img_b64 = base64.b64encode(image_bytes).decode("ascii")
    data_url = f"data:image/png;base64,{img_b64}"

    prompt = (
        f"Décris cette slide pédagogique en {lang}. Identifie : "
        f"(1) le concept principal, (2) les éléments visuels (schémas, "
        f"tableaux, formules, images), (3) le rôle pédagogique. "
        f"Format JSON : {{\"description\":\"...\", \"main_concept\":\"...\", \"language\":\"{lang}\"}}}"
    )

    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{
            "role": "user",
            "content": [
                {"type": "text", "text": prompt},
                {"type": "image_url", "image_url": {"url": data_url}},
            ],
        }],
        max_tokens=300,
        temperature=0.3,
    )
    raw = response.choices[0].message.content
    data = _extract_json(raw) or {}
    return {
        "description": data.get("description", "").strip(),
        "main_concept": data.get("main_concept", "").strip(),
        "language": lang,
        "captured_at": time.time(),
        "model": "gpt-4o-mini",
    }
except Exception as exc:
    log.warning("vision OpenAI failed: %s", exc)
    return None

```

```

def merge_into_slide_content(ocr_text: str, vision_desc: dict, lang: str) -> str:
    """Combine OCR text + vision description for the Q&A handler."""
    if not vision_desc:
        return ocr_text
    desc = vision_desc.get("description", "")
    if not desc:
        return ocr_text
    if lang == "fr":
        return f"{ocr_text}\n\n[Description visuelle] : {desc}"
    return f"{ocr_text}\n\n[Visual description] : {desc}"

```

26. services/nav_dispatcher.py

Rôle : dispatch des actions de navigation vocale ("slide suivante", "ralentis").

```

async def dispatch_nav_action(action: dict, ctx: SessionContext, dialogue, websocket_send):
    """Execute a navigation action issued by the IntentAgent."""
    nav_action = action.get("nav_action", "next")

```

```

nav_target = action.get("nav_target", "")

if nav_action == "next":
    await dialogue.next_section(ctx.session_id)
    await websocket_send({"type": "next_section"})
elif nav_action == "previous":
    await dialogue.prev_section(ctx.session_id)
    await websocket_send({"type": "prev_section"})
elif nav_action == "repeat":
    await websocket_send({"type": "repeat_section"})
elif nav_action == "skip":
    # Skip to next chapter
    ctx.section_index = 0
    ctx.chapter_index += 1
    await dialogue._save(ctx)
    await websocket_send({"type": "skip_chapter"})
elif nav_action == "go_to_concept":
    # Resolve concept name → (chapter, section)
    from pedagogy.knowledge_graph import get_or_build
    kg = get_or_build(get_rag())
    concept = kg.get_concept(nav_target)
    if concept:
        chapter_idx = next(iter(concept.chapter_idxs), 0)
        await websocket_send({"type": "go_to_section",
                               "chapter_idx": chapter_idx, "section_idx": 0})
elif nav_action == "slow_down":
    # Adjust profile speech rate
    from pedagogy.personalization.profile import update_profile
    new_rate = max(Config.SPEECH_RATE_FLOOR,
                    (ctx.preferences.get("speech_rate", 1.0) * Config.SPEECH_RATE_SLOW_DOWN_FACTOR))
    await update_profile(ctx.session_id, {"preferences": {"speech_rate": new_rate}})
    await websocket_send({"type": "speech_rate_changed", "rate": new_rate})
# ... etc

```

27. services/text_turns.py

Rôle : persistence des turns chat (Q&A) pour audit + analytics.

```

async def persist_text_turn(session_id, student_id, course_id, turn_id, role: str,
                             text: str, metadata: dict = None):
    """Persist a chat turn (user OR assistant) :
    1. Append to Redis chat history (rolling)
    2. Store full transcript JSON to media storage
    3. Index in Elasticsearch
    """
    ts = time.time()

    # 1. Redis chat history (bounded, 30j TTL)
    from pedagogy.student_history import append_chat_turn
    await append_chat_turn(student_id, course_id, role, text)

    # 2. Media storage upload (transcript JSON)
    transcript = {
        "session_id": session_id,
        "student_id": student_id,
        "course_id": course_id,
        "turn_id": turn_id,
        "role": role,
        "text": text,
        "timestamp": ts,
        "metadata": metadata or {},
    }
    from storage.media_storage import get_storage
    storage = get_storage()
    obj_name = f"turns/transcripts/{student_id}/{turn_id}_{int(ts*1000)}.json"
    storage.upload_bytes(json.dumps(transcript, ensure_ascii=False).encode("utf-8"),

```

```

        obj_name, "application/json")

# 3. Elasticsearch index (for cross-course search)
from storage.transcript_search import get_search_index
search = get_search_index()
search.index(TranscriptEntry(
    session_id=session_id, language=metadata.get("language", "fr"),
    course_id=course_id, course_title=metadata.get("course_title", ""),
    role=role, text=text, subject=metadata.get("subject", ""),
    timestamp=ts,
))

async def persist_audio_turn(session_id, student_id, course_id, turn_id, audio_bytes, mime, metadata):
    """Same as persist_text_turn but for audio bytes."""
    obj_name = f"turns/answers/{student_id}/{turn_id}_{int(time.time()*1000)}.mp3"
    from storage.media_storage import get_storage
    url = get_storage().upload_bytes(audio_bytes, obj_name, mime)
    metadata["audio_url"] = url
    return url

```

28. services/course_slides.py

Rôle : helper pour résoudre un slide context depuis (course_id, chapter_idx, section_idx).

```

async def load_course_slide_context(course_id: str, chapter_idx: int, section_idx: int) -> Optional[dict]:
    """Returns {slide_path, content, chapter_title, section_title, ...} ou None."""
    try:
        async with AsyncSessionLocal() as db:
            stmt = (
                select(Section, Chapter)
                .join(Chapter, Section.chapter_id == Chapter.id)
                .where(Chapter.course_id == _coerce_uuid(course_id),
                    Chapter.order == chapter_idx,
                    Section.order == section_idx)
            )
            row = (await db.execute(stmt)).first()
            if row is None:
                return None
            section, chapter = row

            return {
                "slide_path": section.image_url,
                "content": section.content,
                "section_title": section.title,
                "chapter_title": chapter.title,
                "chapter_idx": chapter_idx,
                "section_idx": section_idx,
            }
    except Exception as exc:
        log.debug(f"load_course_slide_context failed: {exc}")
        return None

```

29. rag/multimodal_rag.py (key functions)

Pour les fonctions critiques non encore couvertes :

29.1 _make_chat_llm helper

```
def _make_chat_llm(model: str, temperature: float, max_tokens: int):
    """Build a ChatOpenAI repointed at Groq when DISABLE_OPENAI=true."""
    if Config.DISABLE_OPENAI and getattr(Config, "GROQ_API_KEY", None):
        return ChatOpenAI(
            model=getattr(Config, "GROQ_MODEL", "llama-3.3-70b-versatile"),
            api_key=Config.GROQ_API_KEY,
            base_url=getattr(Config, "GROQ_BASE_URL", "https://api.groq.com/openai/v1"),
            temperature=temperature,
            max_tokens=max_tokens,
            max_retries=0,
        )
    return ChatOpenAI(model=model, temperature=temperature, max_tokens=max_tokens, max_retries=0)
```

Utilisé dans `_get_or_create_summary`, `generate_final_answer`, `generate_final_answer_stream`, et la génération de quiz.

29.2 _bm25_alpha (adaptive RRF weight)

```
def _bm25_alpha(self, query: str) -> float:
    """Compute  $\alpha$  for RRF based on query rarity (avg IDF)."""
    if not self.bm25_retriever or not query:
        return 1.0
    tokens = self._tokenize(query.lower())
    if not tokens:
        return 1.0

    # Compute avg IDF for query tokens
    bm25 = self.bm25_retriever.vectorizer
    idfs = []
    for token in tokens:
        if token in bm25.idf_:
            idfs.append(bm25.idf_[token])
    if not idfs:
        return 1.0
    avg_idf = sum(idfs) / len(idfs)

    # Normalize avg_idf to [0, 1]
    L0, HI = 1.5, 6.0
    norm = max(0.0, min(1.0, (avg_idf - L0) / (HI - L0)))
    alpha = 1.0 + 0.32 * norm

    if alpha > 1.1:
        log.info(" ■■ BM25 boost  $\alpha$ =%.2f (query has rare terms)", alpha)
    return alpha
```

29.3 _rrf_fuse

```
def _rrf_fuse(self, vector_docs: list, bm25_docs: list, alpha: float, k: int = 60) -> list:
    """Reciprocal Rank Fusion (Cormack 2009) with adaptive weight."""
    scores = {}
    # Index by content_hash to dedup (vector + bm25 might overlap)
    for rank, doc in enumerate(vector_docs):
        ch = doc.metadata.get("content_hash", id(doc))
        scores[ch] = scores.get(ch, 0) + (2 - alpha) / (k + rank)
    for rank, doc in enumerate(bm25_docs):
        ch = doc.metadata.get("content_hash", id(doc))
        scores[ch] = scores.get(ch, 0) + alpha / (k + rank)

    # Build doc dict from both sources (latest wins)
    doc_by_hash = {}
    for doc in vector_docs + bm25_docs:
        ch = doc.metadata.get("content_hash", id(doc))
        doc_by_hash[ch] = doc

    sorted_hashes = sorted(scores.keys(), key=scores.get, reverse=True)
```

```
return [doc_by_hash[h] for h in sorted_hashes if h in doc_by_hash]
```

29.4 _rerank_with_cross_encoder

```
def _rerank_with_cross_encoder(self, query: str, candidates: list, top_k: int) -> list:
    """Re-rank candidates using bge-reranker-v2-m3 cross-encoder."""
    if not self._reranker or not candidates:
        return candidates[:top_k]

    pairs = [(query, doc.page_content) for doc in candidates]
    scores = self._reranker.compute_score(pairs)          # batch inference

    # Attach scores to docs
    for doc, score in zip(candidates, scores):
        doc.metadata["_rerank_score"] = float(score)

    # Sort desc by rerank score
    sorted_docs = sorted(candidates, key=lambda d: d.metadata.get("_rerank_score", 0), reverse=True)
    return sorted_docs[:top_k]
```

30. rag/embedding_cache.py

Rôle : cache d'embeddings BGE/OpenAI en Redis.

```
class EmbeddingCache:
    def __init__(self, ttl_seconds: int = None):
        self.ttl = ttl_seconds or Config.EMBEDDING_CACHE_TTL      # 24h

    def _key(self, text: str, model: str) -> str:
        digest = hashlib.sha256(f"{model}|{text}".encode("utf-8")).hexdigest()
        return f"embedding_cache:{digest}"

    async def get(self, text: str, model: str) -> Optional[list[float]]:
        from pedagogy.dialogue import get_redis
        r = await get_redis()
        raw = await r.get(self._key(text, model))
        if raw:
            try:
                return json.loads(raw)
            except Exception:
                return None
        return None

    async def set(self, text: str, model: str, embedding: list[float]) -> None:
        from pedagogy.dialogue import get_redis
        r = await get_redis()
        await r.setex(self._key(text, model), self.ttl, json.dumps(embedding))
```

Économise massivement les calls embedding (BGE-m3 sur CPU = 50-200ms par query, multiplié par ~K queries par turn → important).

31. pedagogy/intelligent_ingester.py

Rôle : ingestion multi-format (PDF/DOCX/PPTX/HTML/text) avec OCR + extraction structurée.

```
class IntelligentIngester:
    def __init__(self, ocr_languages: str = "fra+eng", slide_dpi: int = 200):
```

```

self.ocr_languages = ocr_languages
self.slide_dpi = slide_dpi

async def ingest_file(self, file_path: str, media_root: str = "media/courses",
                      course_id: str = "") -> IngestionResult:
    """Dispatch based on file extension."""
    ext = Path(file_path).suffix.lower()
    if ext == ".pdf":
        return await self.ingest_pdf(file_path, media_root, course_id)
    elif ext == ".pptx":
        return await self.ingest_pptx(file_path, media_root, course_id)
    elif ext == ".docx":
        return await self.ingest_docx(file_path, media_root, course_id)
    elif ext in (".html", ".htm"):
        return await self.ingest_html(file_path, course_id)
    elif ext in (".txt", ".md"):
        return await self.ingest_text(file_path, course_id)
    else:
        return await self.ingest_generic(file_path, course_id)

async def ingest_pdf(self, pdf_path: str, media_root: str = "media/courses",
                     course_id: str = "") -> IngestionResult:
    """Step-by-step :
    1. pypdf → texte brut par page
    2. pdf2image → PNG slides (DPI configuré)
    3. pillow + pytesseract → OCR images embedded
    4. unstructured → Tables, FigureCaptions, Titles
    5. Detect language sur premiers 5 pages
    """
    start = time.time()
    result = IngestionResult(file_path=pdf_path, course_id=course_id)

    # Step 1 : pages text
    pages_text = self._extract_pages_text(pdf_path)
    result.total_pages = len(pages_text)
    log.info("■ Pages text : %d pages extracted", result.total_pages)

    # Step 2 : PNG slides
    slide_paths = self._render_slides_to_png(pdf_path, slides_dir)
    result.slide_pngs = [str(p) for p in slide_paths]
    log.info("■ PNG slides : %d generated @ %ddpi", len(slide_paths), self.slide_dpi)

    # Compose pages list
    for i, txt in enumerate(pages_text, start=1):
        slide_path = str(slide_paths[i - 1]) if i - 1 < len(slide_paths) else ""
        result.pages.append({"page_num": i, "text": txt, "slide_path": slide_path})
        if txt.strip():
            result.assets.append(IngestedAsset(asset_type="text", page_num=i, text=txt[:5000]))

    # Step 3 : embedded images + OCR
    try:
        n_imgs, image_assets = self._extract_and_ocr_images(pdf_path, images_dir)
        result.total_images = n_imgs
        result.assets.extend(image_assets)
        log.info("■ Embedded images : %d extracted + OCR'd", n_imgs)
    except Exception as exc:
        log.warning("image extraction failed: %s", exc)

    # Step 4 : unstructured.partition → Tables, FigureCaptions
    try:
        elements = self._run_unstructured(pdf_path)
        result.elements = elements
        for el in elements:
            el_type = type(el).__name__
            page_num = self._safe_page_num(el)
            text = (getattr(el, "text", "") or "").strip()
            if not text:
                continue
            if el_type == "Table":
                result.assets.append(IngestedAsset(

```

```

        asset_type="table", page_num=page_num, text=text[:3000],
        metadata={"category": el_type}))
    result.total_tables += 1
elif el_type == "FigureCaption":
    result.assets.append(IngestedAsset(
        asset_type="caption", page_num=page_num, text=text[:1000],
        metadata={"category": el_type}))
    result.total_captions += 1
except Exception as exc:
    log.warning("unstructured partition failed: %s", exc)

# Step 5 : detect language
all_text = "\n".join(p.get("text", "") for p in result.pages[:5][:3000])
result.language = self._detect_lang(all_text)

result.extraction_time_s = round(time.time() - start, 2)
log.info("■ Ingestion complete : pages=%d images=%d tables=%d captions=%d elapsed=%.2fs",
        result.total_pages, result.total_images, result.total_tables,
        result.total_captions, result.extraction_time_s)
return result

```

32. pedagogy/course_builder.py

Rôle : construit la structure cours (chapters / sections) depuis la sortie d'IntelligentIngestor.

```

class CourseBuilder:
    def __init__(self):
        self.extractor = TextExtractor()
        self.structurer = LocalStructurer(self)
        self.llm = Brain()

    async def build_from_file(self, file_path: str, language: str = "fr",
                             level: str = "lycée") -> dict:
        """Returns course_data dict."""
        # 1. Ingest
        ingester = IntelligentIngestor(ocr_languages="fra+eng")
        ingestion = await ingester.ingest_file(file_path)

        # 2. Resolve section titles via vision LLM ou structural fallback
        slides_with_titles = []
        for page in ingestion.pages:
            title = await self._resolve_section_title(
                content=page["text"],
                parent_title="",
                fallback_index=page["page_num"],
                image_path=page["slide_path"],
                language=language,
            )
            slides_with_titles.append({
                "title": title,
                "content": page["text"],
                "image_url": page["slide_path"],
                "page_index": page["page_num"],
            })

        # 3. Group consecutive slides -> sections of one chapter
        course_data = {
            "title": Path(file_path).stem,
            "language": language,
            "level": level,
            "subject": "general",
            "domain": "general",
            "chapters": [{
                "title": Path(file_path).stem,
                "order": 1,
                "summary": "",
            }]
        }

```

```

        "sections": slides_with_titles,
    }],
    "slides": [s["image_url"] for s in slides_with_titles],
    "file_path": file_path,
}

return course_data

async def save_to_database(self, course_data: dict, db, domain: str = "general") -> str:
    """Insert Course + Chapter + Section rows. Returns course UUID."""
    course = Course(
        title=course_data.get("title"),
        domain=domain,
        subject=course_data.get("subject", "generic"),
        language=course_data.get("language", "en"),
        level=course_data.get("level", "université"),
        description=course_data.get("description", ""),
        file_path=course_data.get("file_path", ""),
    )
    db.add(course)
    await db.flush() # generates course.id (UUID)

    slides = course_data.get("slides", [])
    for ch_data in course_data.get("chapters", []):
        chapter = Chapter(
            course_id=course.id,
            title=ch_data["title"],
            order=ch_data.get("order", 0),
            summary=ch_data.get("summary", ""),
        )
        db.add(chapter)
        await db.flush()

        for i, sec_data in enumerate(ch_data.get("sections", [])):
            section_order = sec_data.get("page_index") or sec_data.get("order") or (i + 1)
            image_url = (sec_data.get("image_url") or "").strip()
            section = Section(
                chapter_id=chapter.id,
                title=sec_data.get("title", ""),
                content=sec_data.get("content", ""),
                order=section_order,
                image_url=image_url,
                summary=sec_data.get("summary", ""),
            )
            db.add(section)

    await db.commit()
    await db.refresh(course)
    return str(course.id)

```

33-37. Knowledge Graph

Voir SMART_TEACHER_FILE_BY_FILE.md (Part 1) §22-23 et SMART_TEACHER_ARCHITECTURE.md §9 — déjà couverts.

Brefs rappels :

- **graph.py** : `KnowledgeGraph` class avec `IdeaNode`, `ConceptInfo`, et 12 méthodes de query
- **builder.py** : `build_from_rag(rag)` + singleton `get_or_build(rag)` avec rebuild on doc count change
- **concepts_loader.py** : `ensure_concepts_loaded(rag, course_id, enrich=False)` lazy load

- **persistence.py** : save/load concepts à disque (`data/kg_concepts_cache.json`) avec invalidation par `n_docs` mismatch
- **concept_from_titles.py** : `ConceptFromTitles.extract` — group by section_title + LLM enrich (skip if `KG_DISABLE_LLM_ENRICH=true`)

38. pedagogy/practice_engine.py

Rôle : génère des questions de pratique (quiz) depuis le contenu du cours.

```
class PracticeEngine:
    def __init__(self, rag, brain):
        self.rag = rag
        self.brain = brain

    async def generate_question(self, concept_name: str, course_id: str,
                               difficulty: str = "balanced",
                               question_type: str = "open") -> Optional[dict]:
        """Generate a practice question from KG concept."""
        from pedagogy.knowledge_graph import get_or_build, ensure_concepts_loaded
        kg = get_or_build(self.rag)
        concepts = await ensure_concepts_loaded(self.rag, course_id, enrich=False)
        concept = kg.get_concept(concept_name)
        if not concept:
            return None

        # Get supporting chunks from KG
        idea_ids = list(concept.idea_ids)[:5]
        chunks = []
        for iid in idea_ids:
            node = kg.get(iid)
            if node:
                chunks.append({"id": iid, "text": node.text, "label": node.label})

        # Build prompt
        prompt = self._build_question_prompt(
            concept_name=concept.canonical_name or concept.display_name,
            chunks=chunks,
            difficulty=difficulty,
            question_type=question_type,
        )
        from ai.llm_router import get_default_router
        raw = get_default_router().invoke(prompt, prefer="openai", max_tokens=400, temperature=0.3)
        question_data = self._parse_question(raw)
        return question_data

    async def grade_attempt(self, question: dict, student_answer: str) -> dict:
        """LLM-graded attempt -> {correct: bool, score: float, feedback: str}."""
        prompt = (
            f"Question : {question['text']}\n"
            f"Réponse attendue : {question['correct_answer']}\n"
            f"Réponse de l'étudiant : {student_answer}\n"
            f"Évalue la réponse. JSON :\n"
            f'{{"correct": true|false, "score": 0..1, "feedback": "..."}}'
        )
        raw = get_default_router().invoke(prompt, prefer="openai", max_tokens=200)
        return _extract_json(raw) or {"correct": False, "score": 0.0, "feedback": "Évaluation impossible"}
```

39. pedagogy/path_recommender.py

Rôle : recommande un chemin d'apprentissage personnalisé.

Utilise FSRs difficulty + bloom_level pour ordonner :

```
BLOOM_DIFFICULTY = {
    "remember": 1,
    "understand": 2,
    "apply": 3,
    "analyze": 4,
    "evaluate": 5,
    "create": 6,
}

def recommend_path(student_id, course_id, target_concept_name: str,
                  mastered_concepts: set[str]) -> list[dict]:
    """Returns ordered list of concepts to study next."""
    from pedagogy.knowledge_graph import get_or_build
    from deps import get_rag
    kg = get_or_build(get_rag())
    target = kg.get_concept(target_concept_name)
    if target is None:
        return []

    # 1. Get prereqs (transitive)
    visited = set()
    queue = deque([target])
    prereqs_in_order = []
    while queue:
        c = queue.popleft()
        if c.name in visited or c.name in mastered_concepts:
            continue
        visited.add(c.name)
        for prereq in kg.prereq_concepts(c.name):
            queue.append(prereq)
            prereqs_in_order.append(c)

    # 2. Sort by Bloom level + difficulty
    def sort_key(c):
        return (BLOOM_DIFFICULTY.get(c.bloom_level, 3), c.score)

    prereqs_in_order.sort(key=sort_key)
    return [
        {
            "name": c.name,
            "canonical": c.canonical_name or "",
            "description": c.description or "",
            "bloom_level": c.bloom_level or "",
            "score": c.score,
        } for c in prereqs_in_order
    ]
```

40. pedagogy/skill_tree.py

Rôle : structure le KG en skill tree visualisable (Cytoscape format).

```
def build_skill_tree(course_id: str) -> dict:
    """Returns Cytoscape-compatible {nodes, edges} dict."""
    from pedagogy.knowledge_graph import get_or_build
    from deps import get_rag
    kg = get_or_build(get_rag())
    concepts = kg.list_concepts(course_id)

    nodes = []
```

```

edges = []
for c in concepts:
    nodes.append({
        "data": {
            "id": c.name,
            "label": c.canonical_name or c.display_name or c.name,
            "score": c.score,
            "bloom": c.bloom_level or "",
            "ideas": len(c.idea_ids),
            "chapter": next(iter(c.chapter_idxs), 0) if c.chapter_idxs else 0,
        }
    })

for c in concepts:
    for prereq in kg.prereq_concepts(c.name):
        weight = kg.count_idea_edges_between(prereq.name, c.name)
        edges.append({
            "data": {
                "source": prereq.name,
                "target": c.name,
                "type": "prereq",
                "weight": weight,
            }
        })
    for example in kg.example_concepts_of(c.name):
        edges.append({
            "data": {
                "source": example.name,
                "target": c.name,
                "type": "example",
                "weight": 1,
            }
        })

return {"nodes": nodes, "edges": edges}

```

41. pedagogy/student_history.py

Rôle : chat history persistente cross-session.

```

async def append_chat_turn(student_id: str, course_id: str, role: str, text: str):
    """Append a chat turn to Redis (bounded to MAX_TURNS)."""
    from pedagogy.dialogue import get_redis
    r = await get_redis()
    key = f"chat::{student_id}:{course_id}"
    entry = json.dumps({"role": role, "text": text, "ts": time.time()})
    await r.rpush(key, entry)
    await r.ltrim(key, -Config.CHAT_HISTORY_MAX_TURNS, -1)  # keep last 30 turns
    await r.expire(key, Config.CHAT_HISTORY_TTL_S)           # 30j TTL

async def load_chat_history(student_id: str, course_id: str, limit: int = 20) -> list[dict]:
    """Load recent chat turns (most recent first)."""
    from pedagogy.dialogue import get_redis
    r = await get_redis()
    key = f"chat::{student_id}:{course_id}"
    raw_entries = await r.lrange(key, -limit, -1)
    history = []
    for raw in raw_entries:
        try:
            history.append(json.loads(raw))
        except Exception:
            continue
    return history

```

42. pedagogy/personalization/engine.py

Rôle : compose le bloc de personnalisation pour le prompt LLM.

```
class PersonalizationEngine:
    @staticmethod
    def build_prompt_prefix(profile: dict, lang: str = "fr") -> str:
        """Returns INSTRUCTION_INTERNE block tuned to student profile."""
        if not profile:
            return ""

        style = profile.get("style", "balanced")          # visual|verbal|kinesthetic
        depth = profile.get("depth", "balanced")          # brief|balanced|deep
        tone = profile.get("tone", "neutral")              # challenging|encouraging|neutral

        if lang == "fr":
            cognitive_styles = {
                "visual": "Privilégie des analogies visuelles, des schémas mentaux, des comparaisons spatiales",
                "verbal": "Utilise un raisonnement verbal explicite, des définitions précises, des chaînes de logique",
                "kinesthetic": "Mets l'étudiant en action : 'imagine que tu manipules X', 'essaie de tracer ça'"
            }
            depths = {
                "brief": "Format CONCIS : 2-3 phrases.",
                "balanced": "Format ÉQUILIBRÉ : 3-4 phrases.",
                "deep": "Format DÉTAILLÉ : 5-6 phrases avec exemple complet.",
            }
            tones = {
                "challenging": "Ton stimulant : pose des sous-questions, pousse l'élève à réfléchir avant de donner sa réponse",
                "encouraging": "Ton encourageant : valide ce que l'étudiant a déjà compris, encourage avant de donner sa réponse",
                "neutral": "Ton factuel et clair.",
            }
            block = (
                "<<INSTRUCTION_INTERNE_NE_PAS_MENTIONNER_DANS_LA_REPONSE>>\n"
                f"Style cognitif à respecter : {cognitive_styles.get(style, cognitive_styles['verbal'])}\n"
                f"{depths.get(depth, depths['balanced'])}\n"
                f"{tones.get(tone, tones['neutral'])}\n"
                "<<FIN_INSTRUCTION_INTERNE>>\n"
            )
        return block
    # English version (similaire)
```

43. pedagogy/personalization/learning_style/bayes.py

Rôle : Bayesian update du posterior VARK.

```
@dataclass
class VARKPosterior:
    visual: float = 0.25
    verbal: float = 0.25
    kinesthetic: float = 0.25
    read_write: float = 0.25

    def normalize(self):
        s = self.visual + self.verbal + self.kinesthetic + self.read_write
        if s > 0:
            self.visual /= s
            self.verbal /= s
            self.kinesthetic /= s
            self.read_write /= s
```

```
def update_posterior(prior: VARKPosterior, signal: str, weight: float = 0.1) -> VARKPosterior:
    """Bayesian update : multiply prior × likelihood, normalize."""
    new = VARKPosterior(**asdict(prior))
    if signal == "visual":
        new.visual *= (1 + weight)
    elif signal == "verbal":
        new.verbal *= (1 + weight)
    elif signal == "kinesthetic":
        new.kinesthetic *= (1 + weight)
    elif signal == "read_write":
        new.read_write *= (1 + weight)
    new.normalize()
    return new

def detect_signal_from_question(question: str, lang: str = "fr") -> Optional[str]:
    """Heuristique : détecter quel style est en jeu dans la question."""
    q = question.lower()
    if any(w in q for w in ["voir", "image", "schéma", "imagine", "graphe", "diagramme"]):
        return "visual"
    if any(w in q for w in ["écrire", "lire", "définir", "définition"]):
        return "read_write"
    if any(w in q for w in ["faire", "manipuler", "essayer", "exemple concret"]):
        return "kinesthetic"
    return "verbal" # default
```

44. pedagogy/personalization/bandit/controller.py

Rôle : orchestre le bandit (start_turn, end_turn) côté business.

```
class BanditController:
    def __init__(self, repo: BanditRepo):
        self.repo = repo

    async def start_turn(self, session_id: str, course_id: str, ctx_features: dict) -> dict:
        """Pick an arm for this turn. Returns {strategy, speech_rate, reasoning, arm_id}."""
        bucket = self._compute_bucket(ctx_features)
        posteriors = await self.repo.get_posteriors_for_bucket(course_id, bucket)

        if not posteriors:
            # Cold start : initialize all 18 arms with Beta(1, 1)
            posteriors = self._init_arms()

        chosen_arm_name = select_arm(posteriors)
        log.info("■ bandit THOMPSON | bucket=%s | %d arms | winner=%s",
                bucket, len(posteriors), chosen_arm_name)

        strategy, speech_rate = chosen_arm_name.split(":")
        reasoning = self._reasoning_for_strategy(strategy)
        log.info("■ bandit START | session=%s ctx_bucket=%s | strategy=%s speech_rate=%s | reasoning=%r",
                session_id[:8], bucket, strategy, speech_rate, reasoning[:80])

        return {
            "strategy": strategy,
            "speech_rate": speech_rate,
            "reasoning": reasoning,
            "arm_id": chosen_arm_name,
            "bucket": bucket,
        }

    async def end_turn(self, session_id: str, course_id: str, decision: dict, outcome: TurnOutcome):
        """Update posterior with reward."""
```

```

reward = compute_reward(outcome)
log.info("bandit.end_turn session=%s arm=%s reward=%.3f ( $\Delta m$ =%.3f confused=%s engaged=%s)",
        session_id[:8], decision["arm_id"], reward,
        outcome.mastery_delta or 0, outcome.confused, outcome.engaged)
await self.repo.update_posterior(course_id, decision["bucket"], decision["arm_id"], reward)

def _compute_bucket(self, ctx_features: dict) -> str:
    """Hash ctx into a bucket string."""
    confusion = ctx_features.get("confusion_score", 0)
    confusion_b = "high" if confusion > 0.7 else "medium" if confusion > 0.3 else "low"

    engagement = ctx_features.get("engagement_score", 0.5)
    eng_b = "engaged" if engagement > 0.65 else "neutral" if engagement > 0.30 else "disengaged"

    mastery = ctx_features.get("avg_mastery", 0.5)
    mast_b = "high" if mastery > 0.7 else "medium" if mastery > 0.4 else "low"

    interaction_density = ctx_features.get("interaction_density", "isolated")

    return f"{confusion_b}|{eng_b}|{mast_b}|{interaction_density}"

def _reasoning_for_strategy(self, strategy: str) -> str:
    REASONS = {
        "socratic": "Adopte un style SOCRATIQUE : pose 1 ou 2 questions guidantes AVANT de donner la r
        "simpler_words": "Réécris en MOTS SIMPLES : remplace le jargon technique par du vocabulaire courant
        "decomposition": "Décompose le concept en SOUS-CONCEPTS plus petits. Présente-les dans l'ordre logi
        "analogy": "Construis ta réponse autour d'une ANALOGIE concrète : compare le concept à un obj
        "recap": "Commence par RÉCAPITULER ce qui a été vu, puis ajoute le nouveau concept en l'att
        "example": "Donne d'abord 1-2 EXEMPLES CONCRETS, puis fais émerger la généralisation depuis c
    }
    return REASONS.get(strategy, "Explique clairement et précisément.")

```

45. pedagogy/personalization/bandit/repo.py

Rôle : persistence Postgres des posteriors.

```

class BanditRepo:
    async def get_posteriors_for_bucket(self, course_id: str, bucket: str) -> dict[str, ArmState]:
        async with AsyncSessionLocal() as db:
            stmt = select(BanditPosterior).where(
                BanditPosterior.course_id == _coerce_uuid(course_id),
                BanditPosterior.bucket == bucket,
            )
            rows = (await db.execute(stmt)).scalars().all()

            if not rows:
                return {}

            result = {}
            total_pulls = 0
            for row in rows:
                result[row.arm_name] = ArmState(alpha=row.alpha, beta=row.beta)
                total_pulls += int(row.alpha + row.beta - 2)

            log.info("bandit loaded – %d posteriors, %d total pulls", len(rows), total_pulls)
            return result

    async def update_posterior(self, course_id: str, bucket: str, arm_name: str, reward: float):
        async with AsyncSessionLocal() as db:
            stmt = select(BanditPosterior).where(
                BanditPosterior.course_id == _coerce_uuid(course_id),
                BanditPosterior.bucket == bucket,
                BanditPosterior.arm_name == arm_name,
            )

```

```

)
row = (await db.execute(stmt)).scalar_one_or_none()
if row is None:
    row = BanditPosterior(
        course_id=_coerce_uuid(course_id),
        bucket=bucket, arm_name=arm_name,
        alpha=1.0, beta=1.0, n_pulls=0,
    )
    db.add(row)
row.alpha += reward
row.beta += (1 - reward)
row.n_pulls += 1
row.last_pull_at = datetime.utcnow()
await db.commit()

```

46. database/init_db.py

Rôle : SQLAlchemy engine + lazy migrations.

```

engine = create_async_engine(
    Config.DATABASE_URL,
    echo=False,
    pool_size=10,
    max_overflow=20,
)

AsyncSessionLocal = async_sessionmaker(
    engine, class_=AsyncSession, expire_on_commit=False,
)

# Lazy migration tables : data-driven ALTER TABLE patches
_LAZY_COLUMN_PATCHES: dict[str, list[tuple[str, str]]] = {
    "student_profiles": [
        ("pace", "VARCHAR(20) DEFAULT 'normal'"),
        ("avg_response_time_s", "DOUBLE PRECISION DEFAULT 0.0"),
        ("confusion_rate", "DOUBLE PRECISION DEFAULT 0.0"),
        ("preferred_explanation_depth", "VARCHAR(20) DEFAULT 'balanced'"),
        ("preferences", "JSONB DEFAULT '{}'::jsonb"),
    ],
    "students": [
        ("student_level", "VARCHAR(20) DEFAULT 'lycée'"),
    ],
}

_LAZY_TYPE_PATCHES: list[tuple[str, str, str, str]] = [
    # (table, column, target_type, USING expression)
    ("learning_sessions", "student_id", "uuid", '"student_id"::uuid'),
    ("interactions", "student_id", "uuid", '"student_id"::uuid'),
    # ...
]

async def run_lazy_migrations():
    """Apply additive + type-conversion patches at boot."""
    async with engine.begin() as conn:
        # Phase 1 : ADD COLUMN IF NOT EXISTS
        for table_name, columns in _LAZY_COLUMN_PATCHES.items():
            result = await conn.execute(
                text("SELECT column_name FROM information_schema.columns WHERE table_name = :t"),
                {"t": table_name})
            existing = {row[0] for row in result}
            for col_name, col_def in columns:
                if col_name in existing:
                    continue

```

```

        stmt = f'ALTER TABLE "{table_name}" ADD COLUMN IF NOT EXISTS "{col_name}" {col_def}'
    try:
        await conn.execute(text(stmt))
        log.info("■ migration: added %s.%s (%s)", table_name, col_name, col_def)
    except Exception as exc:
        log.warning("■■ migration: failed to add %s.%s - %s", table_name, col_name, exc)

# Phase 2 : ALTER COLUMN TYPE
for table_name, col_name, target_type, using_expr in _LAZY_TYPE_PATCHES:
    # Read current udt_name
    result = await conn.execute(
        text("SELECT udt_name FROM information_schema.columns WHERE table_name = :t AND column_name = :c")
        {"t": table_name, "c": col_name})
    row = result.first()
    if row is None or (row[0] or "").lower() == target_type.lower():
        continue
    stmt = f'ALTER TABLE "{table_name}" ALTER COLUMN "{col_name}" TYPE {target_type} USING {using_expr}'
    try:
        await conn.execute(text(stmt))
        log.info("■ migration: converted %s.%s → %s", table_name, col_name, target_type)
    except Exception as exc:
        log.warning("■■ migration: failed to convert %s.%s - %s", table_name, col_name, exc)

```

Ce système évite Alembic pour les ALTER additifs simples. Pour les migrations complexes (RENAME, drops, data migrations), Alembic reste nécessaire.

47. database/models.py

Rôle : SQLAlchemy ORM models.

Tables principales (résumées) :

```

class Student(Base):
    __tablename__ = "students"
    id:          UUID = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    email:       str  = Column(String(255), unique=True, nullable=False, index=True)
    password_hash: str = Column(String(255))
    first_name:  str  = Column(String(100))
    last_name:   str  = Column(String(100))
    preferred_language: str = Column(String(10), default="fr")
    student_level: str = Column(String(20), default="lycée")
    account_level: str = Column(String(20), default="student") # student|teacher|admin
    is_active:   int  = Column(Integer, default=1)
    created_at:  datetime = Column(DateTime, default=datetime.utcnow)

class Course(Base):
    __tablename__ = "courses"
    id:          UUID = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    title:       str  = Column(String(500), nullable=False)
    domain:      str  = Column(String(50), default="general")
    subject:     str  = Column(String(100), default="generic")
    language:    str  = Column(String(10), default="fr")
    level:       str  = Column(String(20), default="université")
    description: str  = Column(Text)
    file_path:   str  = Column(String(500))
    created_at:  datetime = Column(DateTime, default=datetime.utcnow)

class StudentMastery(Base):
    __tablename__ = "student_mastery"
    id:          int  = Column(Integer, primary_key=True)
    student_id:  UUID = Column(UUID(as_uuid=True), nullable=False, index=True)
    course_id:   UUID = Column(UUID(as_uuid=True), nullable=False, index=True)

```

```

idea_id:      str  = Column(String(64), nullable=False, index=True)
attempts:     int  = Column(Integer, default=0)
confusions:   int  = Column(Integer, default=0)
score:        float = Column(Float, default=0.5)
last_seen_at: datetime = Column(DateTime, default=datetime.utcnow)
__table_args__ = (UniqueConstraint("student_id", "course_id", "idea_id"),)

```

```

class ReviewQueue(Base):
    """FSRS state per (student, concept_name)."""
    __tablename__ = "review_queue"
    id: int = Column(Integer, primary_key=True)
    student_id: UUID = Column(UUID(as_uuid=True), nullable=False, index=True)
    concept_name: str = Column(String(255), nullable=False, index=True)
    fsrs_state: dict = Column(JSONB, default={})
    due: datetime = Column(DateTime, index=True)
    last_review: datetime
    review_count: int = Column(Integer, default=0)
    lapse_count: int = Column(Integer, default=0)
    state: str = Column(String(20), default="learning")
    stability: float = Column(Float, default=0.0)
    difficulty: float = Column(Float, default=0.5)
    __table_args__ = (UniqueConstraint("student_id", "concept_name"),)

```

```

class BanditPosterior(Base):
    __tablename__ = "bandit_posteriors"
    id: int = Column(Integer, primary_key=True)
    course_id: UUID = Column(UUID(as_uuid=True), index=True)
    bucket: str = Column(String(255)) # context bucket hash
    arm_name: str = Column(String(50)) # "socratic:slow"
    alpha: float = Column(Float, default=1.0)
    beta: float = Column(Float, default=1.0)
    n_pulls: int = Column(Integer, default=0)
    last_pull_at: datetime = Column(DateTime, default=datetime.utcnow)
    __table_args__ = (UniqueConstraint("course_id", "bucket", "arm_name"),)

```

22 tables au total — voir le fichier complet pour la liste exhaustive.

48. routes/course.py

Rôle : endpoints REST `/course/*`.

48.1 POST `/course/build` (upload PDF)

```

@router.post("/course/build")
async def build_course(
    files: list[UploadFile] = File(...),
    language: str = Form("fr"),
    level: str = Form("lycée"),
    domain: str = Form("general"),
):
    from pedagogy.course_builder import CourseBuilder
    from database.init_db import AsyncSessionLocal

    rag = deps.get_rag()
    if not files:
        raise HTTPException(status_code=400, detail="Aucun fichier fourni")

    results = []
    files_to_index = []
    builder = CourseBuilder()

```

```

for f in files:
    raw_upload_name = (f.filename or "upload.pdf").replace("\\", "/")
    upload_filename = Path(raw_upload_name).name or f"upload_{uuid.uuid4().hex[:8]}.pdf"
    payload = await f.read()

    # 1. Save to temp file
    with tempfile.NamedTemporaryFile(delete=False, suffix=Path(upload_filename).suffix) as tmp:
        tmp.write(payload)
        temp_path = Path(tmp.name)

    # 2. Auto-detect domain/course
    from core.domains_config import auto_detect_course, classify_course_via_llm
    detected_domain, detected_course = auto_detect_course(str(temp_path))
    if detected_domain == "general":
        llm_classification = classify_course_via_llm(str(temp_path), max_pages=2)
        if llm_classification:
            detected_domain = llm_classification["domain"]
            detected_course = llm_classification["course"]

    target_domain = detected_domain if detected_domain != "general" else domain
    target_course = detected_course if detected_course != "generic" else None
    # ... (path resolution + dest file move)

    # 3. Ingest (PDF → pages, slides, OCR, structure)
    from pedagogy.intelligent_ingester import IntelligentIngester
    ingester = IntelligentIngester(ocr_languages="fra+eng")
    ingestion = await ingester.ingest_file(file_path=str(dest), course_id="")

    # 4. Build course structure (LLM titles, group sections)
    course_data = await builder.build_from_ingestion(ingestion, language, level)
    course_id = None
    async with AsyncSessionLocal() as db:
        course_id = await builder.save_to_database(course_data, db, domain=target_domain)

    # 5. Persist ingested assets in DB
    async with AsyncSessionLocal() as db:
        cid_uuid = uuid.UUID(course_id)
        for asset in ingestion.assets:
            db.add(IngestedAssetDB(
                course_id=cid_uuid,
                asset_type=asset.asset_type,
                page_num=asset.page_num,
                text=asset.text[:5000],
                image_path=asset.image_path,
                image_index_in_page=asset.image_index_in_page,
                asset_metadata=asset.metadata,
            ))
        await db.commit()

    files_to_index.append({
        "course_data": course_data, "course_id": course_id,
        "domain": target_domain, "course": target_course,
        "storage_path": str(dest.resolve()),
    })

    results.append({
        "file": upload_filename,
        "course_id": course_id,
        "title": course_data.get("title"),
        "chapters": len(course_data.get("chapters", [])),
        "sections": sum(len(ch.get("sections", [])) for ch in course_data.get("chapters", [])),
        ...
    })

# 6. Index to RAG (Qdrant + KG)
if files_to_index:
    for item in files_to_index:
        rag_ok = rag.run_ingestion_pipeline_from_course_data(
            item["course_data"],
            domain=item["domain"], course=item["course"],
            course_id=item["course_id"], incremental=True,

```

```
)

return {"results": results, "rag_stats": rag.get_stats()}
```

48.2 GET /course/{id}/concept-graph

```
@router.get("/course/{course_id}/concept-graph")
async def concept_graph(course_id: str):
    """Returns Cytoscape format {nodes, edges} for the KG visualization."""
    cache_key = f"concept_graph:{course_id}"
    if cache_key in _concept_graph_cache:
        return _concept_graph_cache[cache_key]

    from pedagogy.skill_tree import build_skill_tree
    from pedagogy.knowledge_graph import ensure_concepts_loaded
    rag = deps.get_rag()

    # Trigger lazy concept extraction with enrich=True (full quality)
    concepts = await ensure_concepts_loaded(rag, course_id, enrich=True)
    if not concepts:
        return {"nodes": [], "edges": []}

    skill_tree = build_skill_tree(course_id)
    _concept_graph_cache[cache_key] = skill_tree
    return skill_tree
```

49. routes/student.py

Rôle : endpoints REST `/student/me/*`.

```
@router.get("/student/me/state")
async def get_student_state(user: dict = Depends(get_current_user)):
    """Returns aggregated state : mastery, confusion, due reviews, snapshot."""
    student_id = uuid.UUID(user["sub"])

    # 1. Total attempts + mastery avg
    async with AsyncSessionLocal() as db:
        stmt = select(
            func.count(StudentMastery.id).label("n"),
            func.avg(StudentMastery.score).label("avg_score"),
            func.sum(StudentMastery.confusions).label("n_confusions"),
        ).where(StudentMastery.student_id == student_id)
        row = (await db.execute(stmt)).first()
        n_attempts = row.n or 0
        mastery_avg = float(row.avg_score or 0.5)
        n_confusions = row.n_confusions or 0

    # 2. Due reviews
    from pedagogy.review_scheduler import ReviewScheduler
    due = await ReviewScheduler.list_due(student_id, limit=10)

    # 3. Knowledge snapshot (last course)
    async with AsyncSessionLocal() as db:
        stmt = select(LearningSession.course_id).where(
            LearningSession.student_id == student_id
        ).order_by(LearningSession.created_at.desc()).limit(1)
        last_course_id = (await db.execute(stmt)).scalar_one_or_none()
    snapshot = None
    if last_course_id:
        from pedagogy.student_knowledge import build_snapshot
        snap = await build_snapshot(str(student_id), str(last_course_id))
        snapshot = {
            "strong": snap.strong_concepts,
```

```

        "weak":            snap.weak_concepts,
        "never_seen":      snap.never_seen_concepts,
        "recently_confused": snap.recently_confused,
    }

# 4. Engagement (computed from session if active)
engagement = {"score": 0.5, "label": "neutral"} # default

return {
    "student_id":      str(student_id),
    "courses_seen":    1 if last_course_id else 0,
    "total_attempts":   n_attempts,
    "mastery_avg":      round(mastery_avg, 2),
    "confusion_count":  n_confusions,
    "due_reviews":      len(due),
    "engagement":       engagement,
    "knowledge_snapshot": snapshot,
}

@router.get("/student/me/learning-style")
async def get_learning_style(user: dict = Depends(get_current_user)):
    student_id = user["sub"]
    profile = await get_or_create_profile(student_id)
    posterior = profile.get("learning_style_posterior", {})
    dominant = max(posterior, key=posterior.get) if posterior else "verbal"
    confidence = posterior.get(dominant, 0.0)
    return {"posterior": posterior, "dominant": dominant, "confidence": confidence}

@router.get("/student/me/practice-due")
async def get_practice_due(user: dict = Depends(get_current_user), limit: int = 20):
    student_id = uuid.UUID(user["sub"])
    from pedagogy.review_scheduler import ReviewScheduler
    due = await ReviewScheduler.list_due(student_id, limit=limit)
    return {"due_count": len(due), "concepts": due}

```

50. observability/logger.py

Rôle : CSV logger pour metrics par turn.

```

class CSVMetricsLogger:
    def __init__(self, filepath: str = None):
        self.filepath = filepath or Config.CSV_LOG_FILE
        self._ensure_header()

    def _ensure_header(self):
        if not os.path.exists(self.filepath):
            with open(self.filepath, "w", newline="", encoding="utf-8") as f:
                writer = csv.writer(f)
                writer.writerow([
                    "timestamp", "session_id", "audio_duration_sec",
                    "stt_time", "llm_time", "tts_time", "total_time",
                    "language", "model_used", "tts_engine_used",
                    "transcription", "subject", "kpi_ok",
                ])

    def log_turn(self, *, audio_duration_sec, stt_time, llm_time, tts_time, total_time,
                  language, model_used, tts_engine_used="edge", tts_model_used="",
                  session_id="", transcription=""):
        meets_kpi = 1 if total_time <= Config.MAX_RESPONSE_TIME else 0
        try:
            with open(self.filepath, "a", newline="", encoding="utf-8") as f:
                writer = csv.writer(f)
                writer.writerow([
                    datetime.utcnow().isoformat(), session_id, audio_duration_sec,

```

```

        stt_time, llm_time, tts_time, total_time,
        language, model_used, tts_engine_used,
        transcription[:200], "", meets_kpi,
    ])
except Exception as exc:
    log.debug("CSV log failed: %s", exc)

```

51. observability/analytics.py

Rôle : ClickHouse analytics OLAP (fallback CSV+RAM si CH down).

```

class AnalyticsEngine:
    def __init__(self):
        self._ch = None          # ClickHouse client
        self._use_ch = False
        self._cache = []         # in-memory fallback
        self._csv_path = ...     # persistent CSV fallback
        # Lazy init at first call

    def _init_clickhouse(self):
        if not Config.CLICKHOUSE_HOST:
            log.info("■ ClickHouse non configuré → analytics CSV + mémoire")
            return False
        try:
            import clickhouse_connect
            self._ch = clickhouse_connect.get_client(
                host=Config.CLICKHOUSE_HOST, port=Config.CLICKHOUSE_PORT,
                database=Config.CLICKHOUSE_DB,
                username=Config.CLICKHOUSE_USER, password=Config.CLICKHOUSE_PASSWORD,
            )
            self._use_ch = True
            log.info("■ ClickHouse connecté : %s:%s/%s",
                Config.CLICKHOUSE_HOST, Config.CLICKHOUSE_PORT, Config.CLICKHOUSE_DB)
            return True
        except Exception as exc:
            log.info("■ ClickHouse non dispo (%s) → analytics CSV + mémoire", exc)
            return False

    def record_event(self, evt: LearningEvent):
        """Triple-write : ClickHouse + CSV + memory."""
        self._cache.append(evt)
        if len(self._cache) > 10000:
            self._cache.pop(0)
        self._write_csv(evt)
        if self._use_ch:
            self._ch_insert(evt)

    def _ch_insert(self, evt):
        try:
            row = asdict(evt)
            row["kpi_ok"] = 1 if row["kpi_ok"] else 0
            row["event_time"] = datetime.fromisoformat(row["event_time"])
            t0 = time.time()
            self._ch.insert("learning_events", [list(row.values())], column_names=list(row.keys()))
            log.info("■ CLICKHOUSE INSERT | table=learning_events lang=%s subj=%s | stt=%.2fs llm=%.2fs total=%s",
                row.get("language"), row.get("subject"),
                float(row.get("stt_time", 0)), float(row.get("llm_time", 0)),
                float(row.get("total_time", 0)),
                (time.time() - t0) * 1000)
        except Exception as exc:
            log.error("■ CLICKHOUSE INSERT FAIL | %s", exc)

```

52. observability/kpi_logger.py

Rôle : tracker spécial pour KPI #1 (interrupt latency).

```
class KPITracker:
    _instance = None

    @classmethod
    def get(cls):
        if cls._instance is None:
            cls._instance = KPITracker()
        return cls._instance

    def __init__(self):
        # Map (session_id, turn_id) → {detected_at, tts_stopped_at}
        self._marks: dict[tuple, dict] = {}
        self._lock = threading.Lock()

    def mark_interrupt_detected(self, session_id: str, turn_id: int):
        """Called by VAD when interrupt is detected client-side."""
        key = (session_id, turn_id)
        with self._lock:
            self._marks[key] = {"detected_at": time.time()}

    def mark_tts_stopped(self, session_id: str, turn_id: int) -> Optional[float]:
        """Called when TTS is effectively cancelled. Returns latency in seconds."""
        key = (session_id, turn_id)
        with self._lock:
            mark = self._marks.get(key)
            if not mark:
                return None
            mark["tts_stopped_at"] = time.time()
            latency = mark["tts_stopped_at"] - mark["detected_at"]
            target = 0.5
            if latency > target:
                log.warning("■■ KPI interrupt: %.0fms > %.0fms target",
                            latency * 1000, target * 1000)
            return latency
```

Fin de Part 2.

Couvre 52 fichiers / sections. Pour le reste (~80 fichiers utilitaires : tests, helpers triviaux, dashboards), voir [SMART_TEACHER_ARCHITECTURE.md](#) et le code source directement.